



Klasyfikacja meczów League of Legends przy ograniczonych danych z pierwszych 10 minut meczu

Michał Wojda* (227753), Sebastian Śmietański* (227914),
Tomasz Teofilak* (227674), Tomasz Żołądek* (230998)

*Wydział Zastosowań Informatyki i Matematyki, Szkoła Główna Gospodarstwa Wiejskiego, Warszawa, Polska

(s227753@sggw.edu.pl, s227914@sggw.edu.pl, s227674@sggw.edu.pl, s230998@sggw.edu.pl)

Otrzymano: 09.06.2026 Zaakceptowano: —

Streszczenie- W pracy przeanalizowano możliwość przewidywania wyniku meczów rankingowych gry League of Legends na podstawie danych dostępnych po pierwszych 10 minutach rozgrywki. Wykorzystano publiczny zbiór danych zawierający 9879 meczów oraz statystyki opisujące m.in. ekonomię drużyn, kontrolę mapy i przebieg walk. Przeprowadzono proces przygotowania danych obejmujący usuwanie wartości odstających, standaryzację zmiennych, redukcję cech silnie skorelowanych oraz agregację wybranych zmiennych z wykorzystaniem analizy PCA. Następnie zastosowano pięć metod klasyfikacji: sieć neuronową MLP, drzewo klasyfikacyjne, algorytm k najbliższych sąsiadów (KNN) oraz dwa modele hybrydowe. Skuteczność modeli oceniono za pomocą miar Accuracy i AUC. Najlepsze wyniki uzyskała sieć neuronowa, osiągając dokładność 73,31% oraz AUC równe 0,811. Otrzymane rezultaty wskazują, że informacje dostępne we wczesnej fazie meczu pozwalają z dobrą skutecznością przewidywać jego końcowy wynik.

Słowa kluczowe: League of Legends, LoL, klasyfikacja, predykcja wyniku meczu, sieci neuronowe, drzewo klasyfikacyjne, KNN, model hybrydowy

Spis treści

1 Wprowadzenie	3
2 Przedmiot badania	4
2.1 Cel i zakres badania	4
2.2 Zbiór danych	5
2.3 Przegląd literatury	5
3 Wstępna analiza danych	6
3.1 Statystyki opisowe	6
3.2 Braki danych	7
3.3 Wartości odstające	7
3.4 Standaryzowanie danych	8
3.5 Usuwanie cech	8
3.5.1 Quasi Stałe	8
3.5.2 Skorelowane prosto, liniowo	9
3.5.3 Skorelowane wielokrotnie, liniowo	11



3.6	Łączenie zmiennych	12
3.6.1	Agregacja	13
3.6.2	PCA	13
4	Analiza końcowego zbioru danych	16
4.1	Wyjaśnienie zmiennych	16
4.2	Charakterystyka zmiennych	17
4.3	Statystyki opisowe	17
4.4	Wizualizacja	18
5	Opisy metod	23
5.1	Sieci neuronowe	23
5.1.1	Opis	23
5.1.2	Funkcja straty	24
5.1.3	Proces uczenia	25
5.2	Drzewo klasyfikacyjne	25
5.2.1	Opis	25
5.2.2	Kryterium podziału	25
5.2.3	Przycinanie drzewa	26
5.3	K najbliższych sąsiadów	26
5.3.1	Opis	26
5.3.2	Miara odległości	26
5.3.3	Dobór parametru k	26
5.3.4	Reguła klasyfikacji	27
5.4	Metoda hybrydowa na bazie średnia arytmetyczna	27
5.5	Metoda hybrydowa na bazie sieci neuronowej	27
5.6	Metody oceniania	28
5.6.1	Macierz pomyłek i miary klasyfikacji	28
5.6.2	Krzywa ROC oraz pole pod krzywą AUC	29
6	Rezultaty	30
6.1	Sieci neuronowe	30
6.2	Drzewo klasyfikacyjne	31
6.3	K najbliższych sąsiadów	33
6.4	Metoda hybrydowa na bazie średniej arytmetycznej	34
6.5	Metoda hybrydowa na bazie sieci neuronowej	35
6.6	Porównanie skuteczności metod	36
6.7	Użycie modeli na syntetycznych danych	36
7	Podsumowanie	36



1. Wprowadzenie

League of Legends jest jedną z najpopularniejszych gier komputerowych typu MOBA (Multiplayer Online Battle Arena), w której dwie pięcioosobowe drużyny rywalizują ze sobą w celu zniszczenia głównej struktury przeciwnika, zwanej Nexusem. Rozgrywka opiera się na współpracy zespołowej, zdobywaniu zasobów oraz stopniowym budowaniu przewagi ekonomicznej i strategicznej nad drużyną przeciwną. Gra toczy się tak długo, aż jedna z drużyn wygra, w grze nie występują remisy.

Na przebieg meczu wpływa wiele wzajemnie powiązanych czynników, takich jak liczba eliminacji, zdobywane złoto, doświadczenie bohaterów, kontrola mapy czy przejmowanie neutralnych obiektów. Szczególnie istotna jest początkowa faza rozgrywki, ponieważ to właśnie w pierwszych minutach meczu budowane są podstawy dalszej przewagi, które często mają wpływ na końcowy rezultat spotkania.

Współczesne gry sieciowe generują duże ilości danych opisujących przebieg rozgrywek, co umożliwia wykorzystanie metod analizy danych oraz uczenia maszynowego do badania zależności występujących podczas meczu. Analiza statystyk z wczesnego etapu gry może dostarczyć informacji o typowych schematach prowadzących do zwycięstwa lub porażki drużyny.

Problem badawczy poruszany w niniejszej pracy dotyczy możliwości przewidywania wyniku meczu rankingowego na podstawie statystyk z pierwszych 10 minut rozgrywki, gdzie przeciętny mecz trwa 30 minut. W pracy wykorzystano dane dotyczące meczów rankingowych w celu zbadania, czy metody klasyfikacji pozwalają przewidywać zwycięstwo lub porażkę drużyny niebieskiej na podstawie cech opisujących początkową fazę meczu. Dodatkowym celem jest określenie, które zmienne mają największy wpływ na skuteczność modeli klasyfikacyjnych.

Wyjaśnienie terminów:

- **Doświadczenie** — Doświadczenie (experience, XP) jest zasobem zdobywanym przez bohaterów podczas rozgrywki głównie poprzez przebywanie w pobliżu eliminowanych przeciwników, takich jak miniony czy potwory z dżungli. Zdobycie doświadczenia pozwala bohaterom awansować na kolejne poziomy, co zwiększa ich statystyki oraz umożliwia rozwijanie umiejętności.
- **Złoto** — Złoto jest podstawową walutą występującą w grze. Może być zdobywane między innymi poprzez eliminowanie minionów, bohaterów przeciwnika, potworów z dżungli oraz niszczenie struktur. Zgromadzone złoto pozwala na zakup przedmiotów zwiększających siłę bohatera.
- **Śmierci, zabójstwa i asysty** — Zabójstwo oznacza wyeliminowanie bohatera drużyny przeciwniej. Drużyna zdobywa wówczas złoto oraz przewagę czasową wynikającą z chwilowej nieobecności przeciwnika na mapie. Śmierć oznacza utratę bohatera na określony czas, co ogranicza możliwości drużyny. Asysta przyznawana jest graczom, którzy uczestniczyli w eliminacji przeciwnika, lecz nie zadali decydującego ciosu.
- **First Blood** — First Blood oznacza pierwsze zabójstwo wykonane w meczu. Zdarzenie to zapewnia dodatkową nagrodę w postaci złota i często umożliwia zdobycie wczesnej przewagi nad przeciwnikiem.
- **Wardy** — Wardy są przedmiotami lub umiejętnościami zapewniającymi wizję wybranego obszaru mapy. Mogą być rozmieszczane przez graczy w celu kontrolowania ruchów przeciwnika oraz zabezpieczania ważnych obiektów. Wrogie wardy mogą zostać wykryte i zniszczone przez drużynę przeciwną, co ogranicza dostęp przeciwników do informacji o mapie.
- **Miniony** — Miniony są jednostkami sterowanymi przez komputer, które regularnie pojawiają się w bazach obu drużyn i poruszają się wyznaczonymi liniami mapy. Eliminowanie minionów stanowi jedno z głównych źródeł złota i doświadczenia podczas wczesnej fazy gry.
- **Miniony z dżungli** — Miniony z dżungli, nazywane również potworami neutralnymi, występują w obszarze dżungli pomiędzy liniami. Ich eliminowanie zapewnia złoto, doświadczenie oraz dodatkowe efekty wzmacniające bohaterów.
- **Smoki** — Smoki są neutralnymi potworami pojawiającymi się okresowo na mapie. Ich pokonanie zapewnia drużynie trwałe premie wzmacniające statystyki bohaterów. Kontrola smoków jest jednym z ważniejszych elementów strategicznych rozgrywki.



- **Heroldy** — Heroldy, określane również jako Rift Herald, są neutralnymi potworami pojawiającymi się we wczesnej i środkowej fazie gry. Po pokonaniu herolda drużyna może przyzwać go na wybranej linii, gdzie pomaga on w niszczeniu struktur przeciwnika.

2. Przedmiot badania

Przedmiotem badania są mecze rankingowe gry League of Legends analizowane na podstawie statystyk pochodzących z pierwszych 10 minut rozgrywki. W pracy skupiono się na zależności pomiędzy wczesną przewagą jednej z drużyn a końcowym wynikiem meczu. Analiza obejmuje zastosowanie metod klasyfikacji w celu sprawdzenia, czy dane dostępne na początkowym etapie gry pozwalają skutecznie przewidywać zwycięstwo lub porażkę drużyny niebieskiej.

2.1. Cel i zakres badania

Celem badania jest klasyfikacja meczów League of Legends na podstawie danych z pierwszych 10 minut rozgrywki przy użyciu kilku metod klasyfikacji, a następnie ocena skuteczności tych metod w przewidywaniu rzeczywistego wyniku meczu, rozumianego jako wygrana lub przegrana drużyny niebieskiej.

Istotnym elementem pracy jest porównanie wyników uzyskanych za pomocą trzech metod klasyfikacji: sieci neuronowych, drzewa klasyfikacyjnego oraz KNN, a także dwóch metod mieszanych, łączących cechy wybranych podejść. Celem analizy jest wskazanie, które z zastosowanych rozwiązań najlepiej przewiduje wynik meczu na podstawie danych dostępnych na wczesnym etapie gry.

Analiza ma charakter predykcyjny i koncentruje się na sprawdzeniu, czy informacje dostępne w początkowej fazie meczu pozwalają na skuteczne przewidywanie jego końcowego rezultatu.

Zakres badania obejmuje:

- analizę surowego zbioru danych,
- przygotowanie zbioru do modeli klasyfikacyjnych,
- analizę końcowego zbioru danych,
- zastosowanie wybranych modeli klasyfikacyjnych i ich walidacja,
- ocenienie i interpretacja wyników klasyfikatorów,
- porównanie skuteczności modeli,
- przetestowanie modeli na syntetycznych danych,

Analizę danych oraz implementację modeli klasyfikacyjnych wykonano przy użyciu języka R. Obliczenia przeprowadzono w środowisku RStudio. Użyte biblioteki: *synthpop*, *psych*, *class*, *caret*, *keras3*, *ggplot2*, *tree*, *factoextra*, *corrplot*, *ggtext*, *dplyr* oraz *tensorflow*.



2.2. Zbiór danych

W pracy wykorzystano publicznie dostępny zbiór danych [7] zawierający statystyki meczów rankingowych gry League of Legends. Dane pochodzą z serwisu Kaggle i zostały pozyskane przy użyciu oficjalnego API udostępnianego przez Riot Games.

Zbiór [7] obejmuje mecze rankingowe rozgrywane na wysokim poziomie rankingowym tj. od rangi Diamond wzwyż, co odpowiada około 5% najlepszych graczy. Dane pochodzą z okresu sprzed około sześciu lat. Autor zbioru nie określa dokładnego regionu serwerowego, z którego zostały pobrane mecze.

Pojedynczy rekord reprezentuje statystyki obu drużyn po upływie pierwszych 10 minut pojedynczego meczu. Dane opisują sytuację ekonomiczną drużyn, przebieg walk, kontrolę mapy oraz realizację celów neutralnych we wczesnej fazie rozgrywki.

Zbiór danych [7] składa się z 9879 rekordów oraz 40 zmiennych. Zmienną decyzyjną jest blueWins, określająca końcowy wynik meczu z perspektywy drużyny niebieskiej. Wartość zmiennej informuje, czy drużyna niebieska odniosła zwycięstwo.

Klasy w zbiorze danych są w przybliżeniu zbalansowane, co oznacza, że liczba zwycięstw drużyny niebieskiej i czerwonej jest podobna.

2.3. Przegląd literatury

Ze względu na ograniczoną liczbę prac dotyczących wczesnej fazy gry w League of Legends, przegląd oparto głównie na dwóch analizach z Kaggle wykorzystujących ten sam zbiór danych.

Pierwsza praca [1] pokazuje, że już w pierwszych 10 minutach meczu pojawiają się istotne różnice statystyczne między drużynami, które mogą wpływać na dalszy przebieg rozgrywki. Wyniki sugerują, że wczesna faza gry dostarcza informacji o potencjalnym wyniku meczu.

Druga analiza [2] wskazuje, że takie zmienne jak liczba zabójstw, asyst, złoto, doświadczenie i zabite stwory dobrze różnicują mecze wygrane i przegrane. Potwierdza to, że przewaga budowana na początku gry często przekłada się na końcowy rezultat.

Trzecia praca [3] zawiera dokładną analizę wszystkich zmiennych oraz ich wpływu na wynik meczu.

Wnioski przedstawione w analizowanych pracach uzasadniają zastosowane w niniejszej pracy podejście badawcze. Dotychczasowe analizy wskazują, że dane z początkowej fazy rozgrywki zawierają istotną informację predykcyjną, a wykorzystanie metod klasyfikacji może umożliwić skuteczne przewidywanie wyniku meczu na podstawie ograniczonego zestawu cech opisujących pierwsze minuty gry.



3. Wstępna analiza danych

3.1. Statystyki opisowe

Zmienna	Średnia	Mediana	Min.	Maks.	Odch. std.	Skośność
blueWardsPlaced	22.29	16.0	5.0	250.0	18.02	4.14
blueWardsDestroyed	2.82	3.0	0.0	27.0	2.17	2.85
blueFirstBlood	0.50	1.0	0.0	1.0	0.50	-0.02
blueKills	6.18	6.0	0.0	22.0	3.01	0.54
blueDeaths	6.14	6.0	0.0	22.0	2.93	0.51
blueAssists	6.65	6.0	0.0	29.0	4.06	0.89
blueEliteMonsters	0.55	0.0	0.0	2.0	0.63	0.69
blueDragons	0.36	0.0	0.0	1.0	0.48	0.57
blueHeralds	0.19	0.0	0.0	1.0	0.39	1.60
blueTowersDestroyed	0.05	0.0	0.0	4.0	0.24	5.59
blueTotalGold	16503.46	16398.0	10730.0	23701.0	1535.45	0.47
blueAvgLevel	6.92	7.0	4.6	8.0	0.31	-0.34
blueTotalExperience	17928.11	17951.0	10098.0	22224.0	1200.52	-0.25
blueTotalMinionsKilled	216.70	218.0	90.0	283.0	21.86	-0.27
blueTotalJungleMinionsKilled	50.51	50.0	0.0	92.0	9.90	0.12
blueGoldDiff	14.41	14.0	-10830.0	11467.0	2453.35	0.03
blueExperienceDiff	-33.62	-28.0	-9333.0	8348.0	1920.37	0.02
blueCSPerMin	21.67	21.8	9.0	28.3	2.19	-0.27
blueGoldPerMin	1650.35	1639.8	1073.0	2370.1	153.54	0.47
redWardsPlaced	22.37	16.0	6.0	276.0	18.46	4.56
redWardsDestroyed	2.72	2.0	0.0	24.0	2.14	2.95
redFirstBlood	0.50	0.0	0.0	1.0	0.50	0.02
redKills	6.14	6.0	0.0	22.0	2.93	0.51
redDeaths	6.18	6.0	0.0	22.0	3.01	0.54
redAssists	6.66	6.0	0.0	28.0	4.06	0.82
redEliteMonsters	0.57	0.0	0.0	2.0	0.63	0.62
redDragons	0.41	0.0	0.0	1.0	0.49	0.35
redHeralds	0.16	0.0	0.0	1.0	0.37	1.85
redTowersDestroyed	0.04	0.0	0.0	2.0	0.22	5.34
redTotalGold	16489.04	16378.0	11212.0	22732.0	1490.89	0.41
redAvgLevel	6.93	7.0	4.8	8.2	0.31	-0.40
redTotalExperience	17961.73	17974.0	10465.0	22269.0	1198.58	-0.28
redTotalMinionsKilled	217.35	218.0	107.0	289.0	21.91	-0.29
redTotalJungleMinionsKilled	51.31	51.0	4.0	92.0	10.03	0.23
redGoldDiff	-14.41	-14.0	-11467.0	10830.0	2453.35	-0.03
redExperienceDiff	33.62	28.0	-8348.0	9333.0	1920.37	-0.02
redCSPerMin	21.73	21.8	10.7	28.9	2.19	-0.29
redGoldPerMin	1648.90	1637.8	1121.2	2273.2	149.09	0.41

Table 1. Statystyki opisowe wszystkich zmiennych.



3.2. Braki danych

W analizowanym zbiorze danych nie stwierdzono braków danych.

3.3. Wartości odstające

Badany zbiór danych z racji bycia pobranym przez API bezpośrednio z serwerów gry nie zawiera błędnych danych, aczkolwiek w celu ograniczenia wpływu wartości odstających na działanie modeli klasyfikacyjnych przeprowadzono proces oczyszczania danych. Za wartości odstające uznano jedynie 1% najwyższych wartości zmiennych [3]:

- blueWardsPlaced
- redWardsPlaced

jako że te zmienne miały największą ilość skrajnie dużych wartości. Dodatkowo, takie wartości świadczą o braku poważnego traktowania gry ze strony drużyny charakteryzującej się wysoką wartością zmiennej *WardsPlaced*.

Dlatego dla każdej wyżej wymienionej cechy wyznaczano górny próg odpowiadający wybranemu percentylowi:

$$Q_g = Q_{1-\frac{p}{100}} \quad (1)$$

gdzie:

- Q_g — górny próg dla danej cechy,
- Q — funkcja percentylowa,
- p — procent najwyższych wartości uznawanych za odstające.

Następnie wszystkie wartości większe od wyznaczonego progu były oznaczane jako brakujące:

$$x = \begin{cases} x, & \text{gdy } x \leq Q_g \\ NA, & \text{gdy } x > Q_g \end{cases} \quad (2)$$

Po oznaczeniu wartości odstających jako brakujące usuwano wszystkie rekordy zawierające przynajmniej jedną wartość NA. Ostatecznie ze zbioru danych usunięto 190 rekordów, co stanowiło w przybliżeniu 1.92% wszystkich obserwacji.

Zastosowana metoda pozwoliła ograniczyć wpływ skrajnych obserwacji na proces uczenia modeli oraz zmniejszyć ryzyko zaburzenia działania algorytmów przez nietypowe wartości.



3.4. Standaryzowanie danych

W celu ujednolicenia rozkładów oraz zakresów wartości poszczególnych cech zastosowano standaryzację danych z wykorzystaniem funkcji `scale()` dostępnej w języku R. Metoda ta polega na przekształceniu wartości zmiennych w taki sposób, aby każda cecha posiadała średnią równą 0 oraz odchylenie standardowe równe 1.

Standaryzacja została wykonana zgodnie ze wzorem:

$$x' = \frac{x - \mu}{\sigma} \quad (3)$$

gdzie:

- x — wartość przed skalowaniem,
- μ — średnia wartość danej cechy,
- σ — odchylenie standardowe danej cechy,
- x' — wartość po standaryzacji.

W wyniku zastosowania standaryzacji wszystkie analizowane cechy numeryczne zostały sprowadzone do wspólnej skali. Pozwoliło to ograniczyć wpływ różnic pomiędzy zakresami wartości poszczególnych zmiennych na działanie modeli klasyfikacyjnych oraz poprawić stabilność procesu uczenia.

3.5. Usuwanie cech

Ta podsekcja będzie poświęcona procesowi doboru cech poprzez usuwanie Quasi stałych oraz usuwanie lub łączenie względnie silnie skorelowanych.

3.5.1. Quasi Stałe

Do detekcji Quasi stałych zmiennych wykorzystaliśmy funkcję z pakietu *Caret* o nazwie `nearZeroVar`. Sprawdza ona, czy istnieją jakieś zmienne, które przyjmują tylko jedną unikalną wartość lub mają bardzo mało unikalnych wartości relatywnie i stosunek częstotliwości najczęściej występującej wartości do częstotliwości drugiej najczęściej występującej wartości jest duży, na co wskazuje sama nazwa funkcji. Po jej wykonaniu otrzymaliśmy następującą listę cech:

- `blueTowersDestroyed`
- `redTowersDestroyed`

Otrzymaliśmy więc cechę "TowersDestroyed" dla obu drużyn. Dodatkowo z tabeli 1 wiemy, że ich mediany wynoszą 0. Nie powinno to zaskakiwać, jako że w pierwszych 10 minutach rozgrywki - względnie początkowej fazie - rzadko się zdarza zniszczenie wieży przeciwnika. W związku z powyższym usuwamy te 2 cechy z naszego data setu.



3.5.2. Skorelowane prosto, liniowo

Cechy skorelowane prosto, liniowo, to takie, pomiędzy którymi istnieje korelacja cecha do cechy. Ergo w tej podsekcji zajmiemy się znajdowaniem takich par i usuwaniu jednej z cech na tej podstawie. Wykorzystamy do tego wbudowaną funkcję w R o nazwie "cor". Za jej pomocą stworzymy macierz korelacji. Wizualizacji dokonamy z wykorzystaniem funkcji "corrplot" z pakietu o tej samej nazwie.

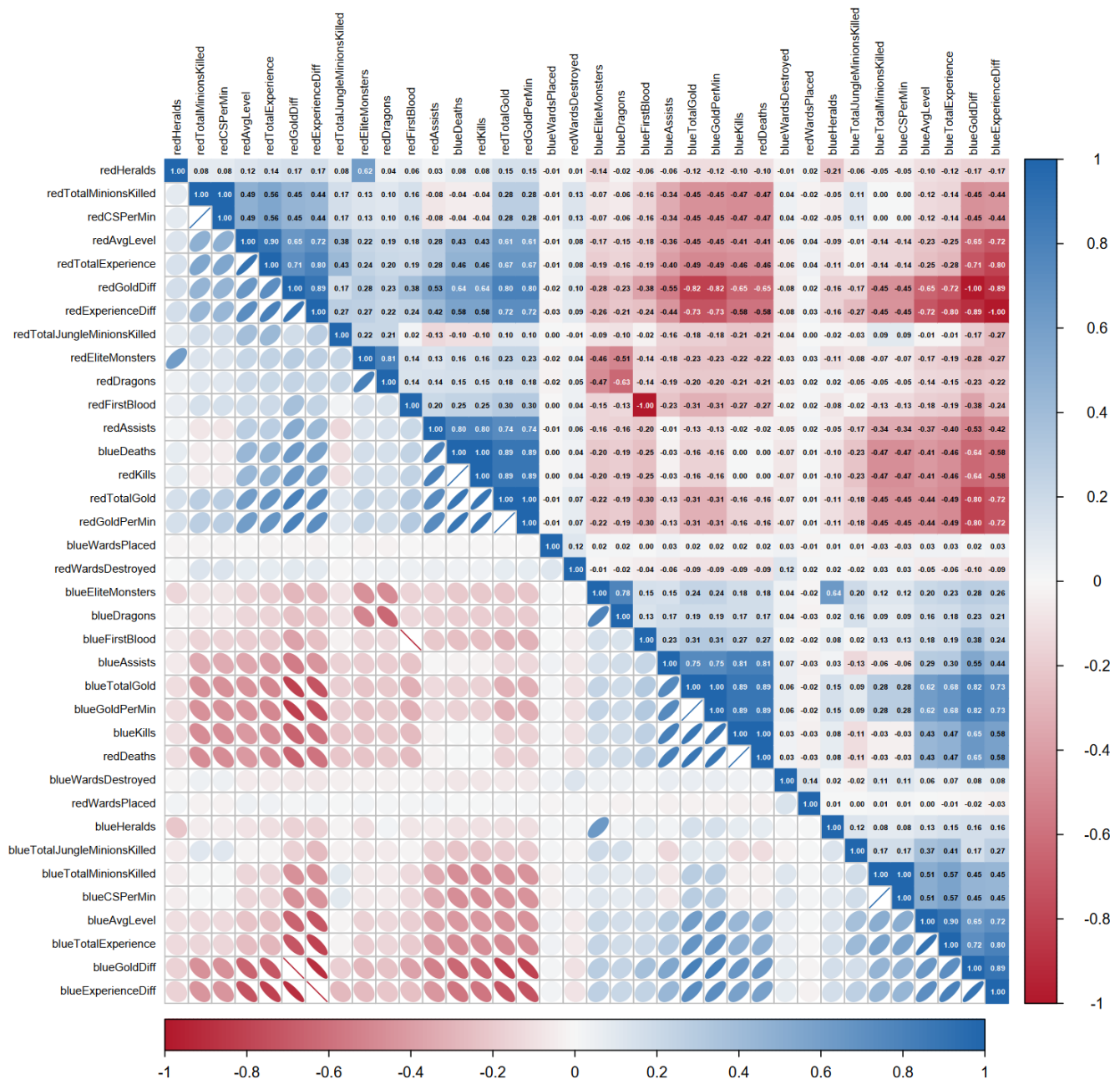


Fig. 1. Macierz korelacji dla wszystkich zmiennych bez quasi stałych 1.



Powyższa macierz przedstawia korelacje liniowe, proste pomiędzy wszystkimi zmiennymi, jakie nam zostały po usunięciu quasi stałych. Na jego podstawie możemy znaleźć bardzo silnie skorelowane, czyli takie, których poziom korelacji jest większy bądź równy 0.9, pary cech:

Cecha A	Cecha B	Korelacja A i B
blueFirstBlood	redFirstBlood	-1
redDeaths	blueKills	1
blueDeaths	redKills	1
blueTotalGold	blueGoldPerMin	1
blueAvgLevel	blueTotalExperience	0.901297
blueTotalMinionsKilled	blueCSPerMin	1
blueGoldDiff	redGoldDiff	-1
blueExperienceDiff	redExperienceDiff	-1
redTotalGold	redGoldPerMin	1
redAvgLevel	redTotalExperience	0.901748
redTotalMinionsKilled	redCSPerMin	1

Table 2. Tabela korelacji silnie skorelowanych zmiennych 1.

Na podstawie powyższego możemy usunąć następujące cech;

- blueFirstBlood
- redDeaths
- blueDeaths
- blueTotalGold
- blueAvgLevel
- blueTotalMinionsKilled
- blueGoldDiff
- blueExperienceDiff
- redTotalGold
- redAvgLevel
- redTotalMinionsKilled



3.5.3. Skorelowane wielokrotnie, liniowo

Dodatkowo, możemy, korzystając z definicji poszczególnych cech, zebrać korelacje wielokrotne, liniowe; innymi słowy takie korelacje, w których to jedna zmienna jest skorelowana liniowo z więcej niż jedną. Takich relacji mamy aż 6 i są one przedstawione poniżej:

- $blueEliteMonsters = blueDragons + blueHeralds$
- $redEliteMonsters = redDragons + redHeralds$
- $blueGoldDiff = blueTotalGold - redTotalGold$ gdzie $blueGoldDiff$ został już usunięty
- $redGoldDiff = redTotalGold - blueTotalGold$
- $blueExperienceDiff = blueTotalExperience - redTotalExperience$ gdzie $blueExperienceDiff$ został już usunięty
- $redExperienceDiff = redTotalExperience - blueTotalExperience$

Na podstawie wyżej wymienionych relacji usuwamy następujące zmienne:

- $blueEliteMonsters$ i $redEliteMonsters$ jako że istnieje potencjalnie istotna różnica pomiędzy zabiciem heralda a smoka i dlatego wolimy zostawić zmienne z wymienionymi skorelowane
- $blueGoldPerMin$ i $redGoldPerMin$ jako że są one liniowo skorelowane odpowiednio z $blueTotalGold$ i $redTotalGold$ oraz przez wzgląd na to, że uznajemy różnicę między nimi za istotniejszą od tych zmiennych samych w sobie
- $blueTotalExperience$ i $redTotalExperience$ jako że uznajemy różnicę między nimi za istotniejszą od tych zmiennych samych w sobie



3.6. Łączenie zmiennych

Po poprzedniej podsekcji zostały nam cechy przedstawione na narysowanej poniżej macierzy korelacji:

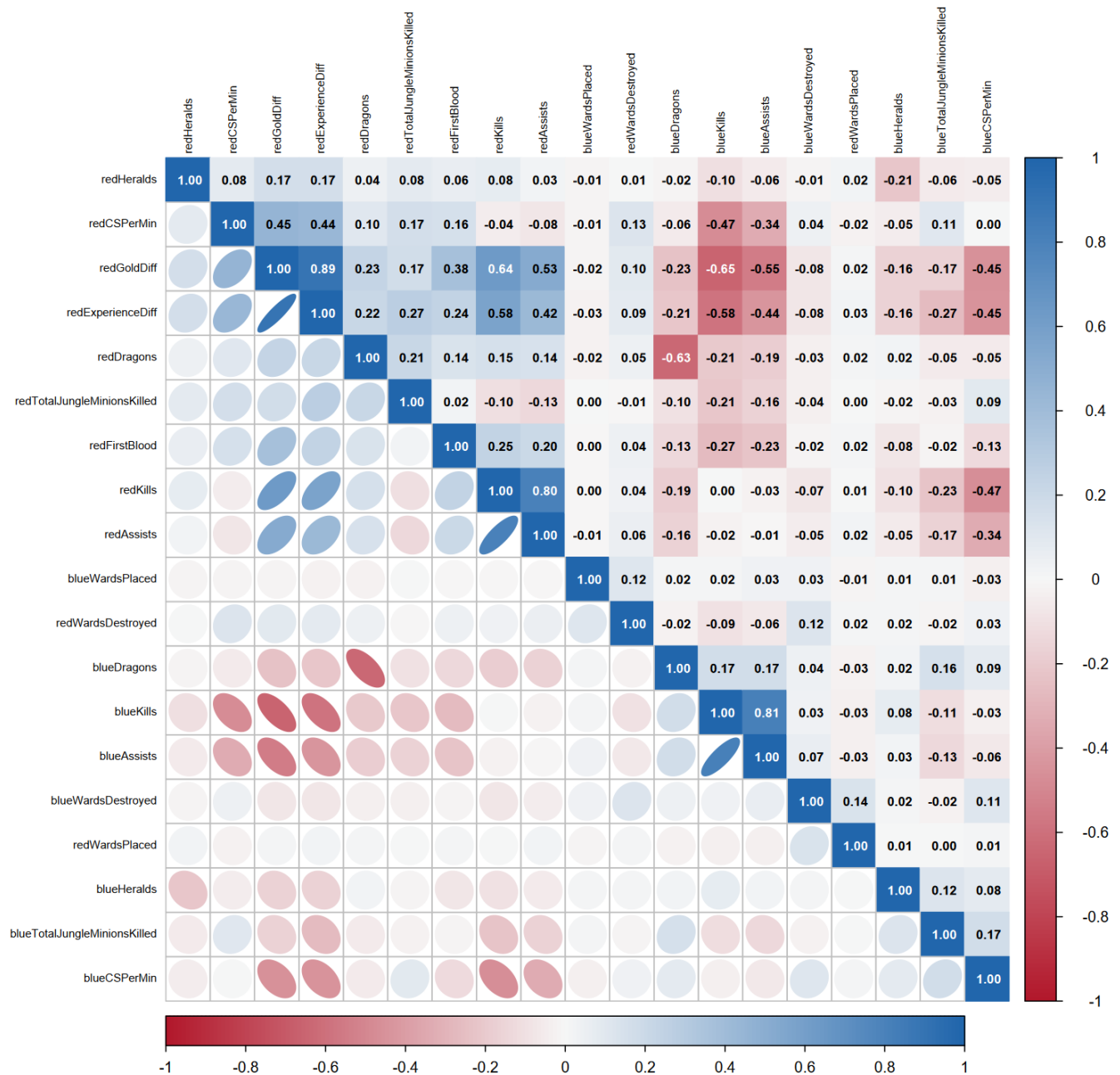


Fig. 2. Macierz korelacji dla wszystkich zmiennych po usuwaniu liniowo skorelowanych



3.6.1. Agregacja

Jak można wyczytać z powyższej macierzy, mamy 3 pary silnie skorelowanych cech. Spośród nich 2 to następujące pary:

Cecha A	Cecha B	Korelacja A i B
redKills	redAssists	0.8
blueKills	blueAssists	0.81

Table 3. Tabela korelacji silnie skorelowanych zmiennych 2.

Te cechy zagrągujemy liniowo w jedną, dla każdej drużyny, wedle następującej reguły [3]:

$$KA = Kills + Assists \quad (4)$$

Tym samym otrzymujemy następujące nowe cechy:

- blueKA
- redKA

3.6.2. PCA

Po agregacji została nam następująca para cech silnie skorelowanych:

Cecha A	Cecha B	Korelacja A i B
redGoldDiff	redExperienceDiff	0.89

Table 4. Tabela korelacji silnie skorelowanych zmiennych 3.

Do zagregowania tych zmiennych wykorzystamy metodę PCA. Poniżej przedstawimy, opis tej metody:

Niech macierz danych będzie opisana jako:

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad (5)$$

gdzie n oznacza liczbę obserwacji, a p liczbę zmiennych. Metoda PCA przekształca pierwotny zbiór p skorelowanych zmiennych w nowy zbiór zmiennych, zwanych **głównymi składowymi**, które są wzajemnie nieskorelowane i uporządkowane według malejącej wariancji.

W pierwszym kroku dane są zwykle centrowane, a w przypadku zmiennych mierzonych w różnych skalach również standaryzowane. Następnie wyznacza się macierz kowariancji:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}, \quad (6)$$

lub, w przypadku danych standaryzowanych, macierz korelacji.

Kolejnym etapem jest wyznaczenie wartości własnych i wektorów własnych macierzy $\mathbf{S}$:

$$\mathbf{S} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad (7)$$

gdzie λ_k oznacza k -tą wartość własną, a $\mathbf{v}_k$ odpowiadający jej wektor własny. Wektory własne wyznaczają kierunki głównych składowych, natomiast wartości własne informują o ilości wariancji wyjaśnianej przez każdą składową.



Główne składowe wyznacza się jako liniowe kombinacje zmiennych pierwotnych:

$$\mathbf{z}_k = \mathbf{X}\mathbf{v}_k. \quad (8)$$

Pierwsza główna składowa wyjaśnia największą możliwą część wariancji danych, druga – największą część pozostałej wariancji, przy zachowaniu ortogonalności względem pierwszej, itd.

Redukcja wymiarowości polega na wyborze pierwszych m składowych, gdzie $m < p$, tak aby zachować jak największą część całkowitej zmienności danych. Udział wariancji wyjaśnionej przez k -tą składową można obliczyć jako:

$$\frac{\lambda_k}{\sum_{j=1}^p \lambda_j}. \quad (9)$$

Z kolei skumulowany udział wyjaśnionej wariancji dla pierwszych m składowych wynosi:

$$\frac{\sum_{k=1}^m \lambda_k}{\sum_{j=1}^p \lambda_j}. \quad (10)$$

W praktyce wybiera się taką liczbę składowych, aby skumulowana wariancja osiągała założony poziom, np. 80%, 90% lub 95%. Dzięki temu możliwe jest zastąpienie dużej liczby zmiennych mniejszym zbiorem nowych cech, co upraszcza analizę, zmniejsza złożoność modeli oraz ogranicza problem współliniowości.

W skrypcie będziemy bazować na wbudowanej funkcji "prcomp" i do wizualizacji rzutu wykorzystamy funkcję "fviz_pca_biplot" z biblioteki factoextra. Poniżej przedstawiony jest wykres po zastosowaniu PCA.

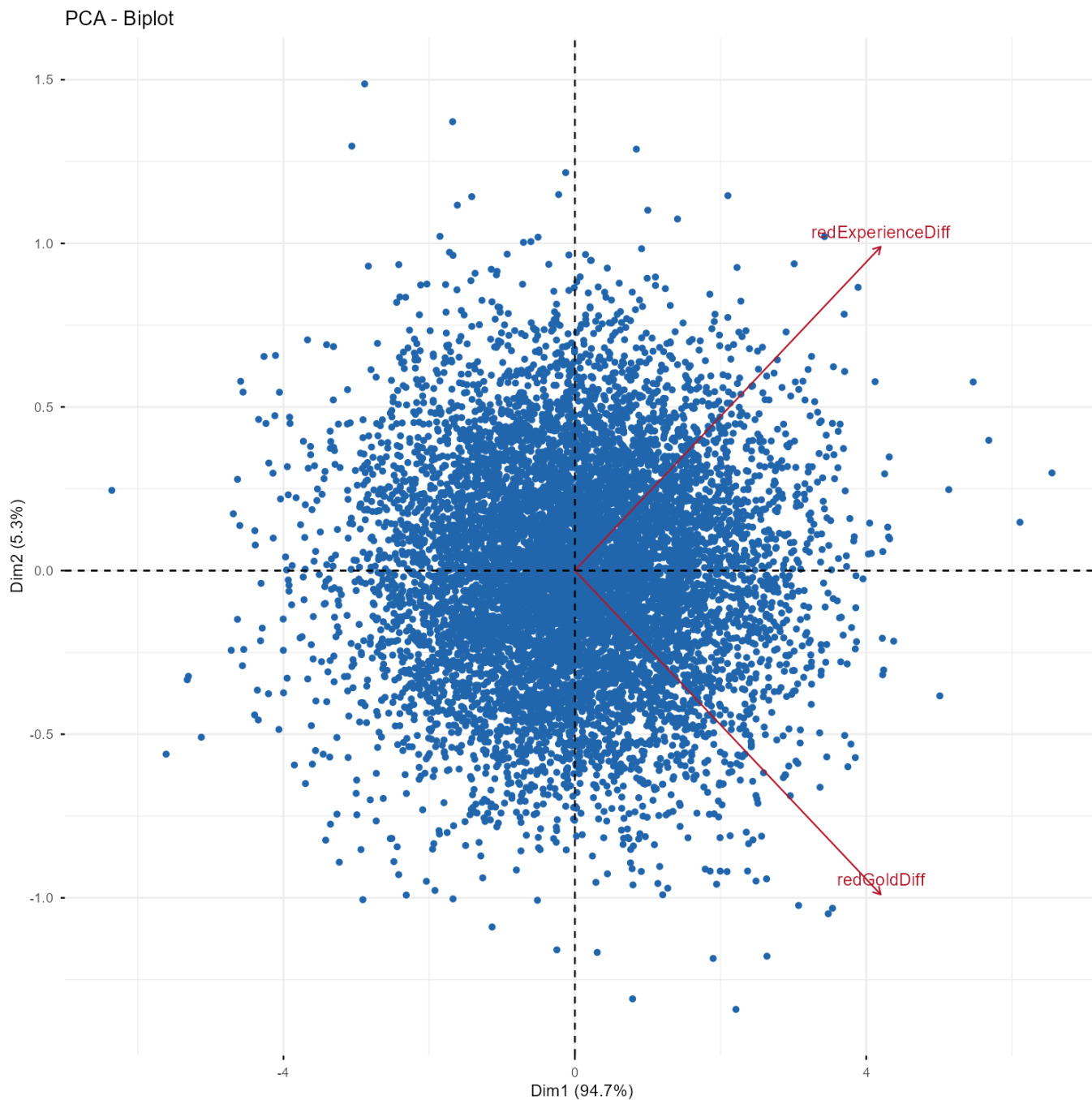


Fig. 3. Wykres PCA dla cech redGoldDiff i redExperienceDiff

Jak na załączonym wyżej obrazku widać, nowa cecha, roboczo nazwana *Dim1*, odpowiada za aż około 94.7% wariancji. Stąd też możemy uznać, że taka agregacja będzie dobrą formą wyeliminowania silnej korelacji. Od tego momentu zmienna robocza nazwana *Dim1* będzie miała nazwę *diff_PCA_Component_1*.

Po wykonaniu wszystkich kroków usuwania silnie i bardzo silnie skorelowanych cech możemy przedstawić jak ostatecznie wygląda macierz korelacji. Dodatkowo macierz uwzględniła cechę *blueWins*, aby dodatkowo ukazać jak silnie jest ona skorelowana ze zmiennymi, które ostatecznie zostały.



4. Analiza końcowego zbioru danych

Po wstępnym przetworzeniu zbioru danych zostało nam 9689 rekordów oraz 16 zmiennych (nie licząc zmiennej przewidywanej).

4.1. Wyjaśnienie zmiennych

- **blueWardsPlaced** — Ilość wardów postawionych przez drużynę niebieską.
- **blueWardsDestroyed** — Ilość wardów zniszczonych przez drużynę niebieską.
- **blueDragons** — Liczba smoków zabitych przez drużynę niebieską.
- **blueHeralds** — Liczba Heroldów zabitych przez drużynę niebieską.
- **blueTotalJungleMinionsKilled** — Łączna ilość zabitych minionów z dżungli przez drużynę niebieską.
- **blueCSPerMin** — Ilość zabitych minionów przez drużynę niebieską w przeliczeniu na minute.
- **redWardsPlaced** — Ilość wardów postawionych przez drużynę czerwoną.
- **redWardsDestroyed** — Ilość wardów zniszczonych przez drużynę czerwoną.
- **redFirstBlood** — Informacja o zdobyciu pierwszego zabójstwa (First Blood) przez drużynę czerwoną.
- **redDragons** — Liczba smoków zabitych przez drużynę czerwoną.
- **redHeralds** — Liczba Heroldów zabitych przez drużynę czerwoną.
- **redTotalJungleMinionsKilled** — Łączna ilość zabitych minionów z dżungli przez drużynę czerwoną.
- **redCSPerMin** — Ilość zabitych minionów przez drużynę czerwoną w przeliczeniu na minute.
- **blueKA** — Łączna ilość zabójstw i asyst drużyny niebieskiej.
- **redKA** — Łączna ilość zabójstw i asyst drużyny czerwonej.
- **diff_PCA_Component_1** — Nowa zmienna utworzona w procesie PCA ze zmiennych *redGoldDiff* i *redExperienceDiff*.



4.2. Charakterystyka zmiennych

Zmienne opisujące ilość zabitych smoków i heroldów powinny być zmiennymi numerycznymi jednak przez ograniczoną naturę badanego zbioru danych przyjmują jedynie wartości 0 lub 1. Z tego powodu zostały one opisane jako zmienne binarne.

Zmienna	Typ	Skala	Charakter
blueWardsPlaced	Numeryczna	Ilościowa	Skokowa
blueWardsDestroyed	Numeryczna	Ilościowa	Skokowa
blueDragons	Binarna	Nominalna	Skokowa
blueHeralds	Binarna	Nominalna	Skokowa
blueTotalJungleMinionsKilled	Numeryczna	Ilościowa	Skokowa
blueCSPerMin	Numeryczna	Ilościowa	Ciągła
redWardsPlaced	Numeryczna	Ilościowa	Skokowa
redWardsDestroyed	Numeryczna	Ilościowa	Skokowa
redFirstBlood	Binarna	Nominalna	Skokowa
redDragons	Binarna	Nominalna	Skokowa
redHeralds	Binarna	Nominalna	Skokowa
redTotalJungleMinionsKilled	Numeryczna	Ilościowa	Skokowa
redCSPerMin	Numeryczna	Ilościowa	Ciągła
blueKA	Numeryczna	Ilościowa	Skokowa
redKA	Numeryczna	Ilościowa	Skokowa
diff_PCA_Component_1	Numeryczna	Ilościowa	Ciągła

Table 5. Charakterystyka końcowych zmiennych.

4.3. Statystyki opisowe

Zmienna	Średnia	Mediana	Min.	Maks.	Odch. std.	Skośność
blueWardsPlaced	0.00	-0.37	-1.16	6.02	1.00	2.94
blueWardsDestroyed	0.00	0.08	-1.30	11.10	1.00	2.85
blueDragons	0.36	0.00	0.00	1.00	0.48	0.57
blueHeralds	0.19	0.00	0.00	1.00	0.39	1.60
blueTotalJungleMinionsKilled	0.00	-0.05	-5.09	4.19	1.00	0.12
blueCSPerMin	0.00	0.06	-5.79	3.03	1.00	-0.26
redWardsPlaced	0.00	-0.38	-1.12	6.11	1.00	2.81
redWardsDestroyed	0.00	-0.34	-1.27	9.92	1.00	2.95
redFirstBlood	0.50	0.00	0.00	1.00	0.50	0.02
redDragons	0.41	0.00	0.00	1.00	0.49	0.35
redHeralds	0.16	0.00	0.00	1.00	0.37	1.86
redTotalJungleMinionsKilled	0.00	-0.03	-4.71	4.05	1.00	0.23
redCSPerMin	0.00	0.03	-5.04	3.26	1.00	-0.29
blueKA	0.00	-0.12	-1.90	5.66	1.00	0.69
redKA	0.00	-0.12	-1.92	4.54	1.00	0.63
diff_PCA_Component_1	0.00	0.00	-4.61	4.75	1.00	-0.03

Table 6. Statystyki opisowe końcowych zmiennych.



4.4. Wizualizacja

Poniżej przedstawiono wizualizacje zmiennych po wstępnej analizie danych z pominięciem skalowania dla zachowania czytelności wykresów. Dla zmiennych skokowych posiadających więcej niż 55 unikatowych wartości wygenerowano histogramy zamiast barplotów.

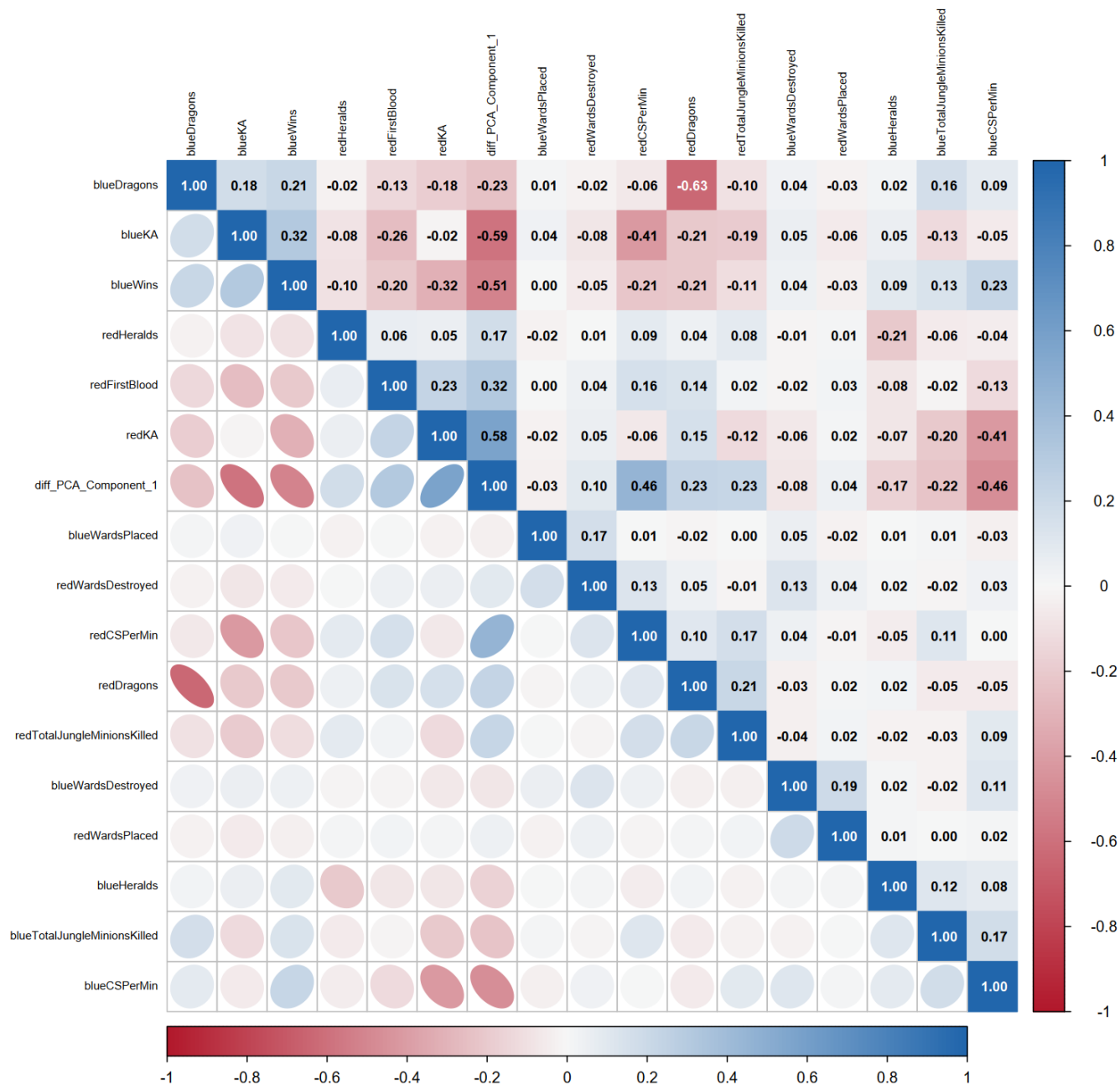


Fig. 4. Macierz korelacji dla ostatecznych zmiennych oraz dla zmiennej przewidywanej *blueWins*.

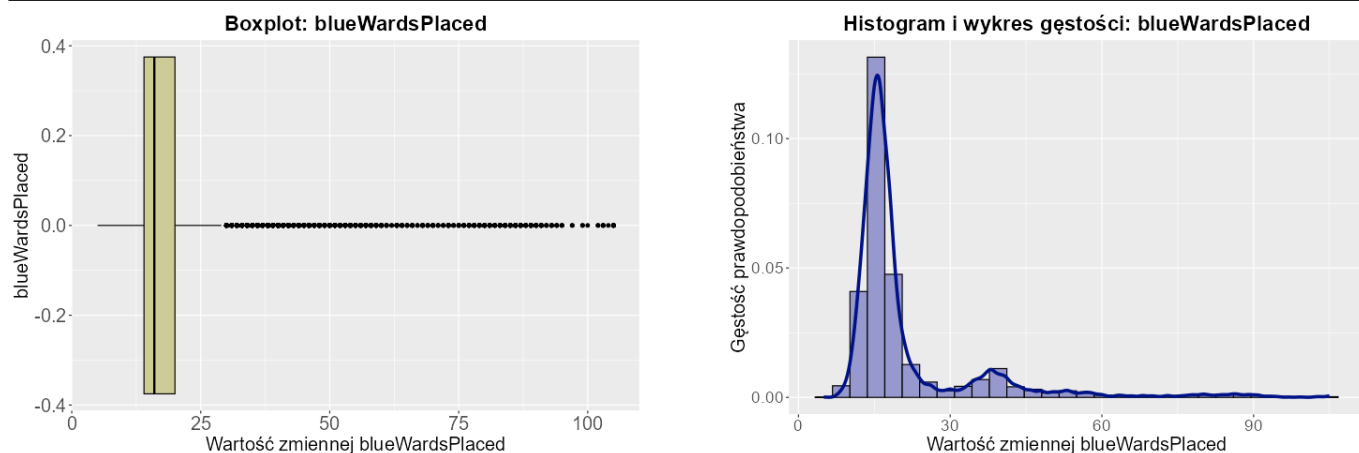


Fig. 5. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueWardsPlaced.

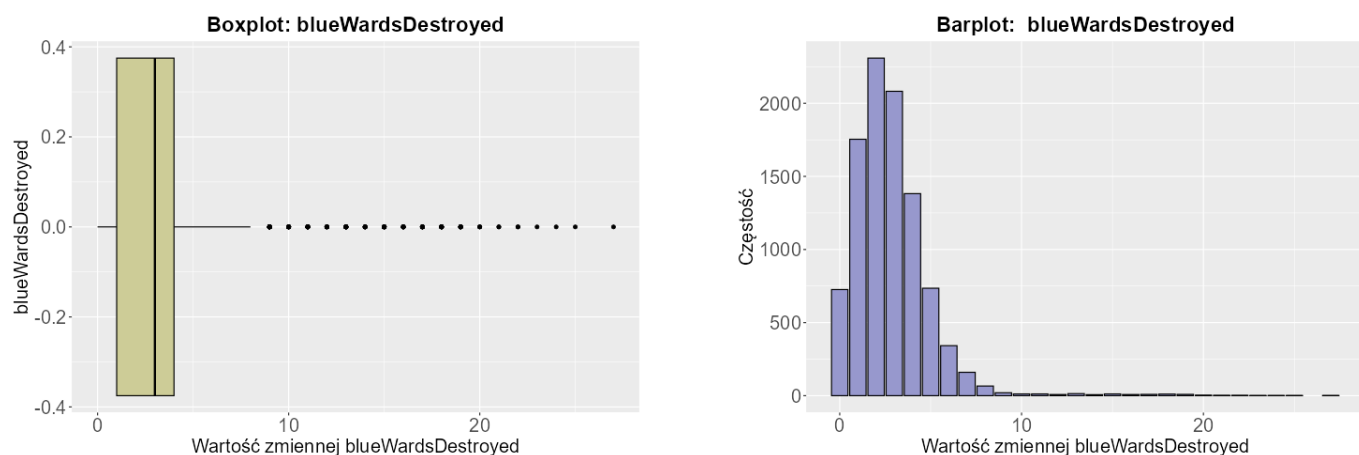


Fig. 6. Boxplot oraz barplot dla zmiennej blueWardsDestroyed.



Fig. 7. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueTotalJungleMinionsKilled.

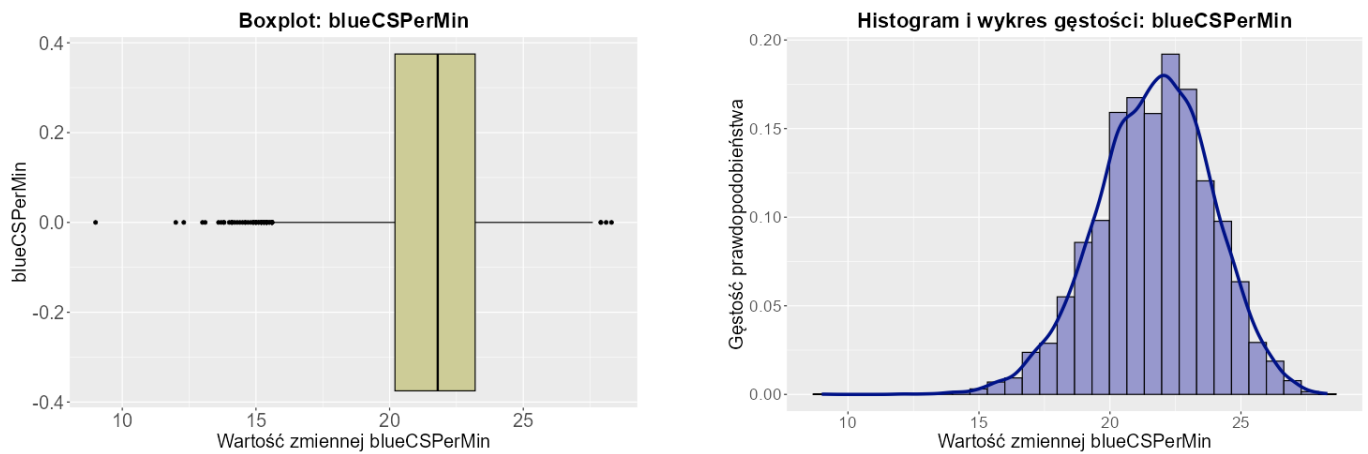


Fig. 8. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej `blueCSPerMin`.

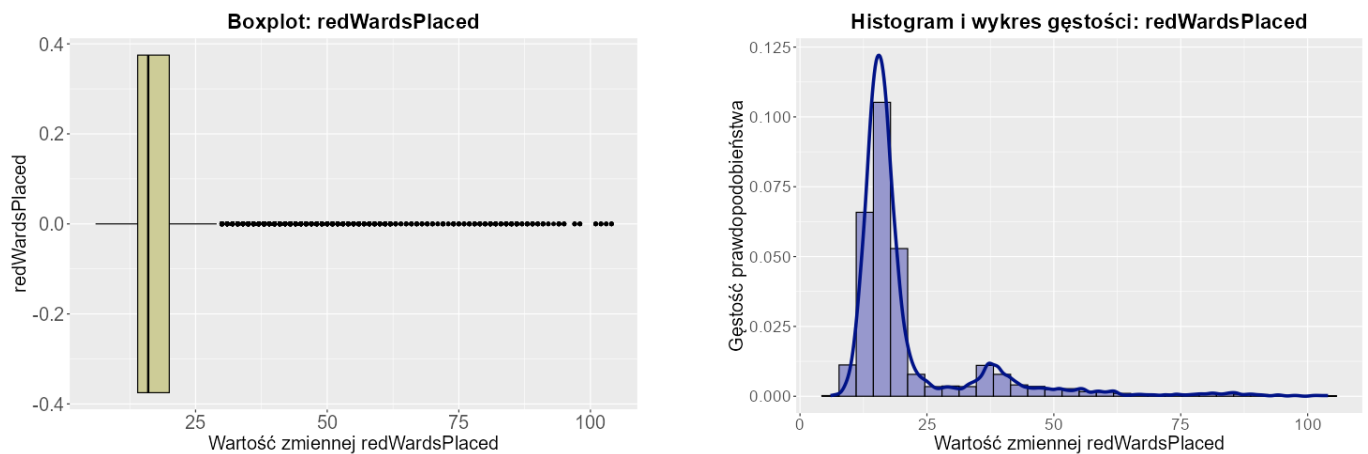


Fig. 9. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej `redWardsPlaced`.

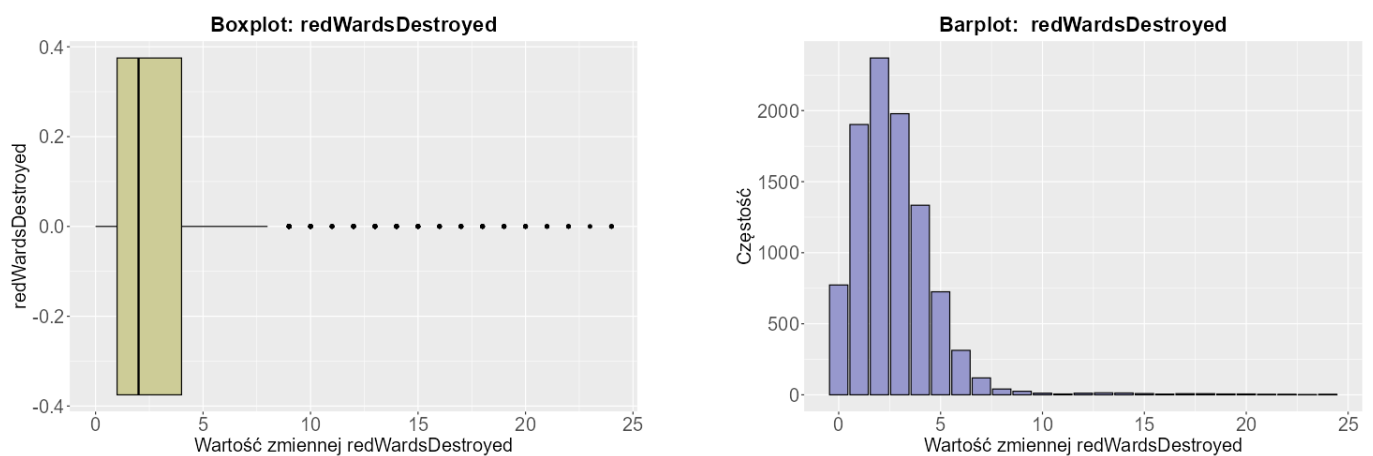


Fig. 10. Boxplot oraz barplot dla zmiennej `redWardsDestroyed`.

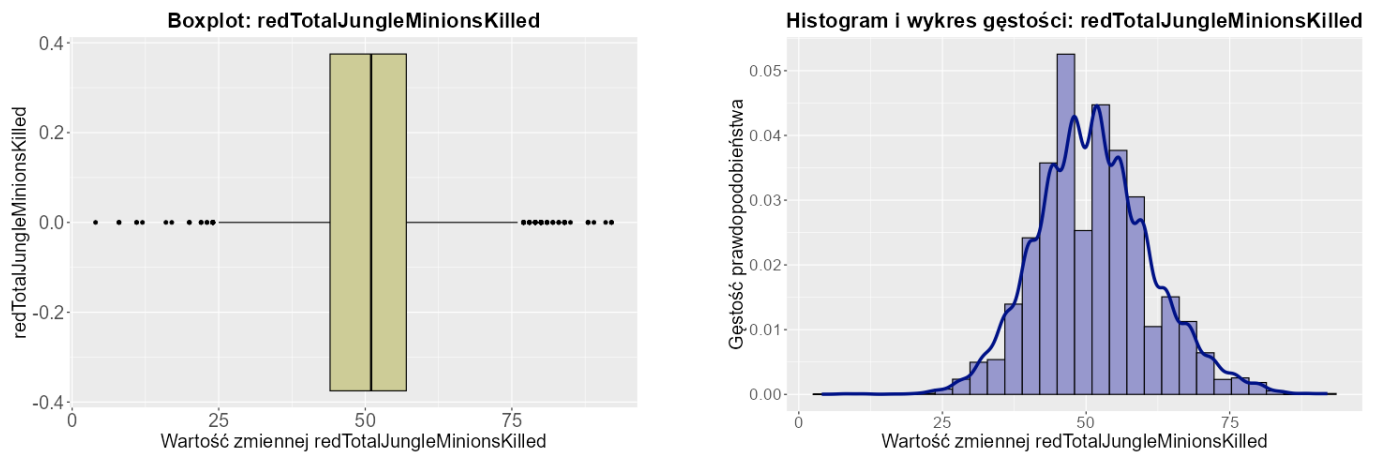


Fig. 11. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej `redTotalJungleMinionsKilled`.

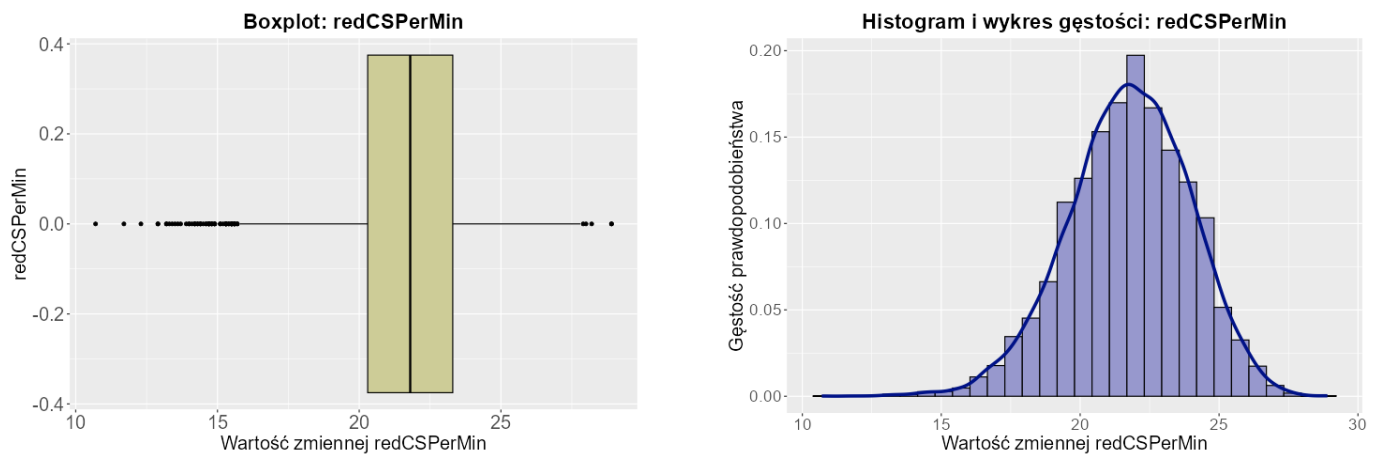


Fig. 12. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej `redCSPerMin`.

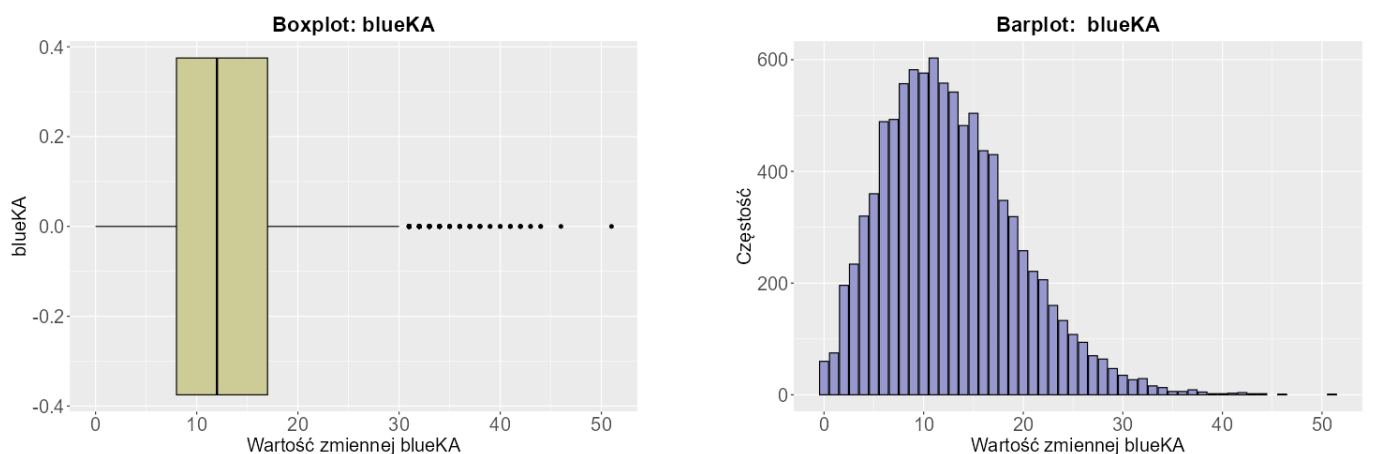


Fig. 13. Boxplot oraz barplot dla zmiennej `blueKA`.

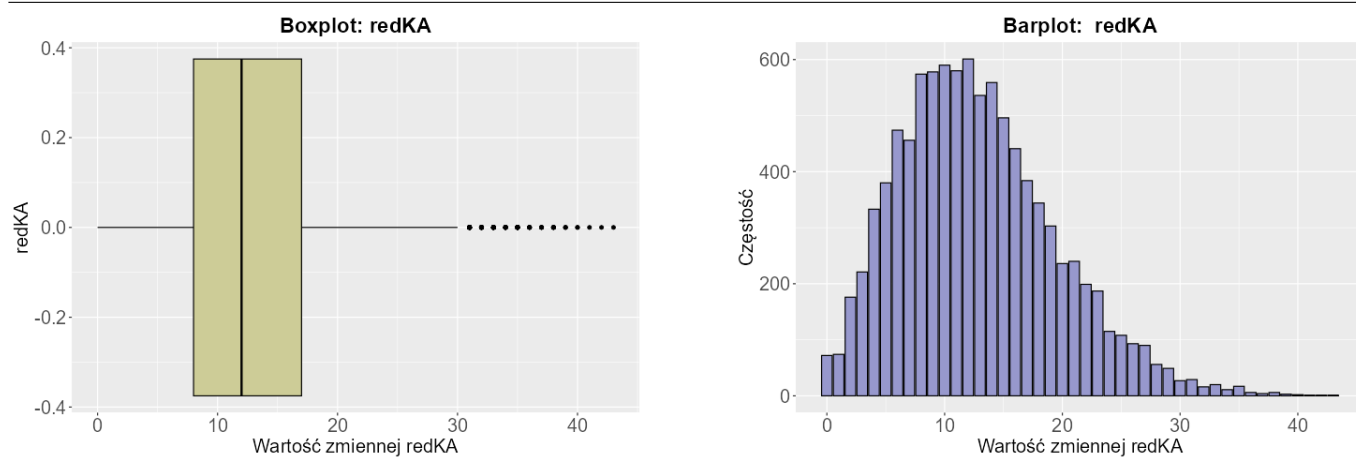


Fig. 14. Boxplot oraz barplot dla zmiennej `redKA`.

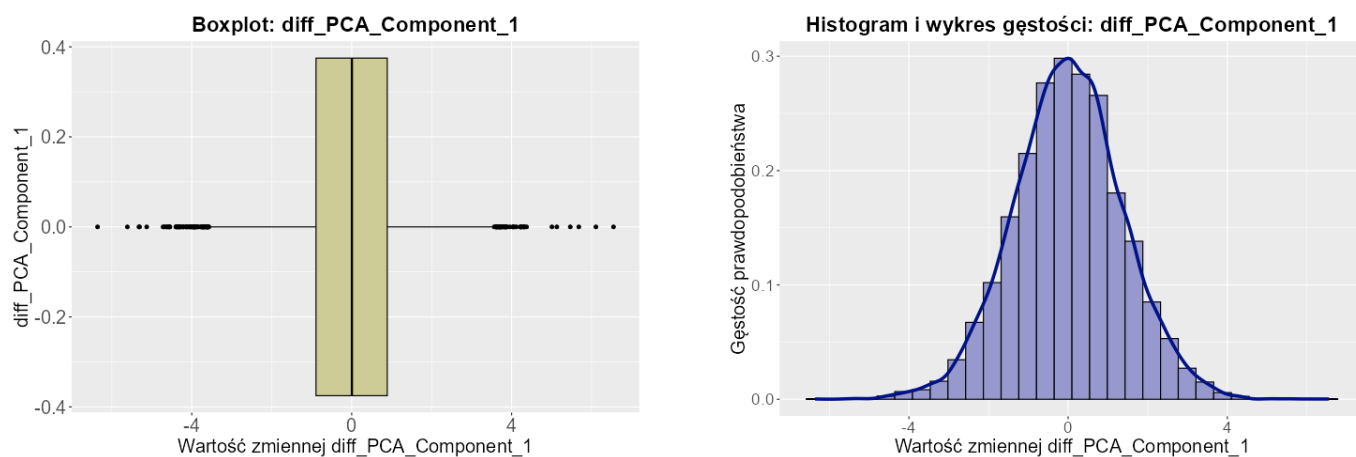


Fig. 15. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej `diff_PCA_Component_1`.

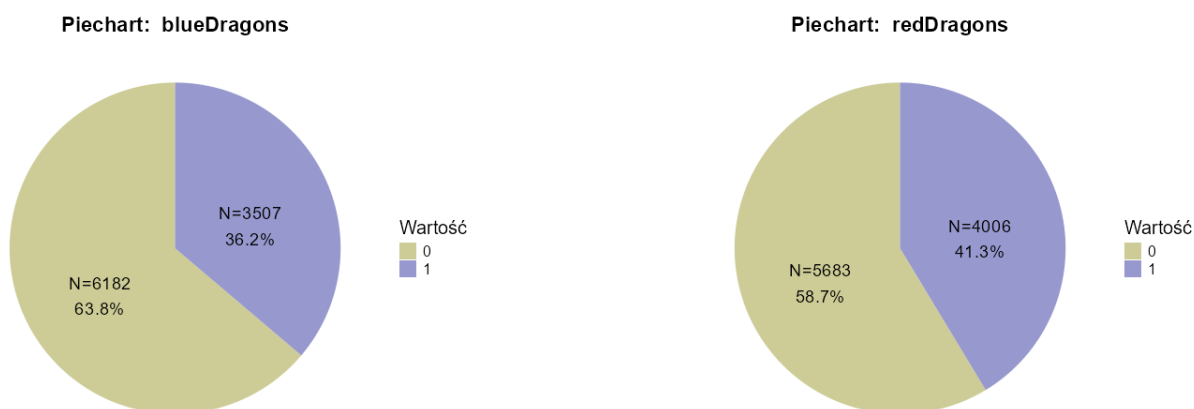


Fig. 16. Piecharty dla zmiennych `blueDragons` oraz `redDragons`.

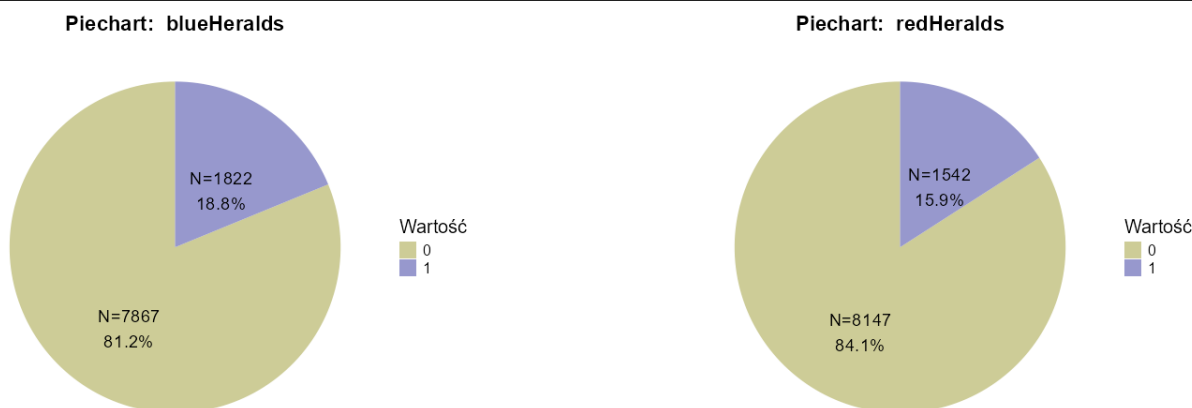


Fig. 17. Piecharty dla zmiennych blueHeralds oraz redHeralds.

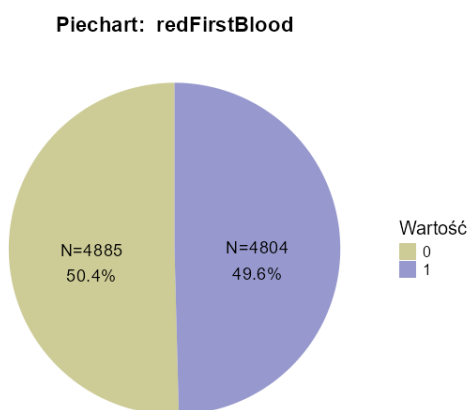


Fig. 18. Piechart dla zmiennej redFirstBlood.

5. Opisy metod

5.1. Sieci neuronowe

5.1.1. Opis

Sieci neuronowe [4] są modelami uczenia maszynowego inspirowanymi działaniem biologicznych sieci neuronowych. Składają się z połączonych ze sobą neuronów tworzących kolejne warstwy przetwarzające dane wejściowe. Dzięki zastosowaniu nieliniowych funkcji aktywacji sieci neuronowe są zdolne do modelowania złożonych zależności pomiędzy zmiennymi.

W analizie wykorzystano wielowarstwową sieć neuronową typu MLP (Multi-Layer Perceptron) zaimplementowaną przy użyciu bibliotek `keras3` oraz `tensorflow`. Zadaniem modelu było przewidywanie wartości zmiennej `blueWins`, określającej końcowy wynik meczu z perspektywy drużyny niebieskiej.

Dla wektora wejściowego $\mathbf{x}$ predykcja modelu może zostać zapisana jako:

$$\hat{y} = \sigma(f(\mathbf{x}, \theta)) \quad (11)$$

gdzie:



- $\mathbf{x}$ – wektor cech wejściowych,
- θ – zbiór parametrów modelu,
- $f(\cdot)$ – transformacja realizowana przez warstwy ukryte,
- $\sigma(\cdot)$ – funkcja sigmoidalna.

Funkcja sigmoidalna przyjmuje postać:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (12)$$

i zwraca wartość z przedziału $[0, 1]$, interpretowaną jako prawdopodobieństwo przynależności do klasy pozytywnej.

Nasza funkcja ostatecznie przyjmuje postać:

$$p = P(Y | X_1 = x_1, \dots, X_n = x_n) = \frac{1}{1 + e^{-(\theta_0 + \sum_{i=1}^n \theta_i x_i)}} \quad (13)$$

Gdzie:

- p – predykcja wygranej,
- n – ilość cech.

Zastosowana architektura składała się z czterech warstw:

- warstwy wejściowej zawierającej 16 cech,
- pierwszej warstwy ukrytej zawierającej 128 neuronów z funkcją aktywacji ELU, gdzie funkcja aktywacji ELU to:

$$f(x) = \begin{cases} e^x - 1 & x \leq 0 \\ x & x > 0 \end{cases} \quad (14)$$

- drugiej warstwy ukrytej zawierającej 64 neurony z funkcją aktywacji Softplus, gdzie funkcja aktywacji Softplus to:

$$f(x) = \ln(1 + e^x) \quad (15)$$

- warstwy wyjściowej zawierającej jeden neuron z funkcją sigmoidalną.

W celu ograniczenia zjawiska przeuczenia pomiędzy warstwami zastosowano mechanizm *dropout* z parametrem $p = 0.5$, polegający na losowym wyłączeniu części neuronów podczas procesu uczenia.

5.1.2. Funkcja straty

Proces uczenia polegał na minimalizacji funkcji straty binary cross-entropy:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (16)$$

gdzie:

- N – liczba obserwacji,



- y_i – rzeczywista wartość klasy,
- $\hat{y}_i$ – prawdopodobieństwo przewidziane przez model.

Minimalizacja funkcji straty była realizowana przy użyciu algorytmu RMSprop, który adaptacyjnie dostosowuje współczynnik uczenia na podstawie historii gradientów.

5.1.3. Proces uczenia

Dane zostały losowo podzielone na zbiór treningowy (80% obserwacji) oraz testowy (20% obserwacji). Model trenowano przez 100 epok przy wielkości batcha wynoszącej 100 obserwacji. Co każdą epokę ponownie dzielimy zbiór na podzbiór testowy i uczący.

Po zakończeniu procesu uczenia wyznaczono prawdopodobieństwa przynależności do klasy pozytywnej dla obserwacji ze zbioru testowego. Otrzymane wartości zostały następnie wykorzystane do oceny jakości klasyfikacji przy użyciu miar opisanych w sekcji dotyczącej metod oceniania.

5.2. Drzewo klasyfikacyjne

5.2.1. Opis

Drzewo decyzyjne [5] jest metodą klasyfikacji wykorzystującą strukturę hierarchiczną składającą się z węzłów decyzyjnych oraz liści reprezentujących klasy. Celem algorytmu jest podział przestrzeni cech na obszary możliwie jednorodne pod względem zmiennej decyzyjnej.

Proces budowy drzewa rozpoczyna się od umieszczenia wszystkich obserwacji w węźle korzeniowym. Następnie wykonywane są kolejne podziały zbioru danych na podstawie wartości wybranych cech. W każdym kroku wybierany jest taki podział, który prowadzi do największego zwiększenia jednorodności klas w otrzymanych podzbiorach.

Algorytm budowy drzewa można przedstawić w postaci następujących kroków:

1. Umieszczenie wszystkich obserwacji w węźle korzeniowym.
2. Wyznaczenie najlepszego podziału dla każdej dostępnej cechy.
3. Wybór podziału maksymalizującego jakość rozdzielania klas.
4. Utworzenie nowych węzłów potomnych.
5. Powtarzanie kroków 2–4 aż do spełnienia kryterium zatrzymania.

W analizie wykorzystano drzewo klasyfikacyjne służące do przewidywania wartości zmiennej `blueWins`, określającej końcowy wynik meczu z perspektywy drużyny niebieskiej.

Dane zostały losowo podzielone na zbiór treningowy (70% obserwacji) oraz testowy (30% obserwacji). Model został wytrenowany na zbiorze treningowym, natomiast jego skuteczność oceniono na niezależnym zbiorze testowym. Dodatkowo podczas procesu przycinania drzewa wykorzystano walidację krzyżową w celu wyznaczenia optymalnego rozmiaru modelu i ograniczenia zjawiska przeuczenia.

5.2.2. Kryterium podziału

Podczas budowy drzewa zastosowano kryterium dewiancji, wykorzystywane przez bibliotekę `tree`. Dla pojedynczego węzła dewiancja definiowana jest jako:

$$D = -2 \sum_{i=1}^K n_i \ln(p_i) \quad (17)$$

gdzie:



- K – liczba klas,
- n_i – liczba obserwacji należących do klasy i ,
- p_i – prawdopodobieństwo wystąpienia klasy i w analizowanym węźle.

Optymalny podział jest wybierany w taki sposób, aby maksymalnie zmniejszyć całkowitą dewiancję drzewa.

5.2.3. Przycinanie drzewa

Drzewa decyzyjne są podatne na przeuczenie (*overfitting*), szczególnie gdy ich struktura staje się zbyt rozbudowana. W celu ograniczenia tego zjawiska zastosowano proces przycinania drzewa (*pruning*).

Przycinanie polega na usuwaniu części gałęzi, które nie wnoszą istotnej informacji predykcyjnej. W pracy optymalny rozmiar drzewa został wyznaczony z wykorzystaniem walidacji krzyżowej. Spośród analizowanych struktur wybierano model minimalizujący błąd klasyfikacji na danych walidacyjnych.

5.3. K najbliższych sąsiadów

5.3.1. Opis

Metoda k najbliższych sąsiadów (ang. *k-Nearest Neighbours*, KNN) [6] należy do grupy metod klasyfikacji opartych na podobieństwie obserwacji. Klasyfikacja nowego obiektu odbywa się na podstawie klas przypisanych do k najbardziej podobnych obserwacji ze zbioru treningowego.

Dla każdej obserwacji testowej wyznaczani są jej najbliżsi sąsiedzi w przestrzeni cech, a następnie przypisywana jest klasa najczęściej występująca wśród znalezionych sąsiadów. Metoda nie tworzy jawnego modelu podczas etapu uczenia, dlatego zaliczana jest do grupy metod leniwych (ang. *lazy learning*).

W analizie wykorzystano algorytm KNN zaimplementowany w bibliotekach *caret* oraz *class*.

5.3.2. Miara odległości

Kluczowym elementem algorytmu KNN jest wyznaczenie odległości pomiędzy obserwacjami. W analizie wykorzystano odległość euklidesową:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2} \quad (18)$$

gdzie:

- $\mathbf{x}$ oraz $\mathbf{y}$ – porównywane obserwacje,
- p – liczba cech,
- x_j, y_j – wartości j -tej cechy.

Na podstawie obliczonych odległości wybieranych jest k najbliższych sąsiadów analizowanej obserwacji.

5.3.3. Dobór parametru k

Jednym z najważniejszych parametrów algorytmu jest liczba sąsiadów k . Zbyt małe wartości mogą prowadzić do nadmiernego dopasowania modelu do danych treningowych, natomiast zbyt duże mogą powodować utratę zdolności rozróżniania klas.

W celu wyznaczenia optymalnej wartości parametru k zastosowano dziesięciokrotną walidację krzyżową. Analizowano wyłącznie nieparzyste wartości parametru z przedziału od 1 do 101:

$$k \in \{1, 3, 5, \dots, 101\}$$

Jako wartość końcową przyjęto parametr zapewniający najwyższą dokładność (*accuracy*) klasyfikacji podczas walidacji krzyżowej.

5.3.4. Reguła klasyfikacji

Klasa nowej obserwacji wyznaczana jest na podstawie głosowania większościowego wśród k najbliższych sąsiadów:

$$\hat{y} = \arg \max_{c \in C} \sum_{i \in N_k} I(y_i = c) \quad (19)$$

gdzie:

- C – zbiór możliwych klas,
- N_k – zbiór k najbliższych sąsiadów,
- $I(\cdot)$ – funkcja wskaźnikowa przyjmująca wartość 1, gdy warunek jest spełniony, oraz 0 w przeciwnym przypadku.

5.4. Metoda hybrydowa na bazie średnia arytmetyczna

W podejściu hybrydowym wykorzystano trzy modele predykcyjne: sieć neuronową, drzewo decyzyjne oraz algorytm k -NN. Każdy z modeli generuje prawdopodobieństwo przynależności do klasy pozytywnej. Następnie wyniki te są agregowane poprzez uśrednienie, co pozwala uzyskać końcową prognozę w postaci jednego wskaźnika decyzyjnego:

$$S = \frac{P_{NN} + P_{KNN} + P_{TREE}}{3} \quad (20)$$

gdzie:

- P_{NN} — prawdopodobieństwo uzyskane z sieci neuronowej,
- P_{KNN} — prawdopodobieństwo uzyskane z KNN,
- P_{TREE} — prawdopodobieństwo uzyskane z drzewa decyzyjnego.

Na podstawie wartości S wyznaczana jest klasa predykcji z wykorzystaniem progu decyzyjnego 0.5:

$$\hat{y} = \begin{cases} 1 & \text{dla } S \geq 0.5 \\ 0 & \text{dla } S < 0.5 \end{cases} \quad (21)$$

W tym podejściu nie stosowano dodatkowego podziału na zbiór treningowy i testowy, ponieważ wszystkie modele bazowe były już wcześniej trenowane i oceniane na zbiorze testowym. W związku z tym, w metodzie hybrydowej cały dostępny zbiór traktowany jest jako zbiór testowy i wykorzystywany wyłącznie do agregacji wyników modeli składowych.

5.5. Metoda hybrydowa na bazie sieci neuronowej

Model hybrydowy oparty na sieci neuronowej stanowi rozszerzenie bazowej sieci MLP. W przeciwieństwie do modelu bazowego, który wykorzystuje bezpośrednio cechy wejściowe zbioru danych, w podejściu hybrydowym jako dane wejściowe zastosowano prawdopodobieństwa klas wygenerowane przez trzy niezależne klasyfikatory: sieć neuronową MLP, algorytm k najbliższych sąsiadów (KNN) oraz drzewo klasyfikacyjne.



Uzyskane wartości prawdopodobieństw zostały połączone w nowy zbiór cech, który następnie wykorzystano do trenowania końcowej sieci neuronowej pełniącej rolę klasyfikatora nadrzędnego. Takie podejście należy do metod typu *stacking*, w których predykcje modeli bazowych stanowią wejście dla modelu wyższego poziomu.

Pod względem architektury model hybrydowy zachowuje taką samą strukturę jak bazowa sieć neuronowa, tj. dwie warstwy ukryte oraz warstwę wyjściową. Ze względu na mniejszą liczbę cech wejściowych zmodyfikowano jednak liczbę neuronów w warstwach ukrytych. W modelu bazowym zastosowano odpowiednio 128 i 64 neurony, natomiast w modelu hybrydowym liczby te zredukowano do 32 i 8 neuronów. Pozostawiono te same funkcje aktywacji, warstwy regularyzacyjne typu Dropout, funkcję kosztu `binary_crossentropy` oraz optymalizator `RMSprop`.

W porównaniu z bazową siecią neuronową model hybrydowy operuje na trzech cechach wejściowych odpowiadających predykcjom modeli bazowych, zamiast na pełnym zbiorze cech opisujących obserwacje. Celem takiej konstrukcji jest połączenie mocnych stron różnych algorytmów klasyfikacyjnych i uzyskanie lepszej jakości predykcji niż w przypadku pojedynczego modelu.

5.6. Metody oceniania

Ocena jakości modeli klasyfikacyjnych stanowi istotny element analizy, ponieważ pozwala na porównanie skuteczności zastosowanych metod oraz określenie ich przydatności w kontekście rozważanego problemu predykcji wyniku meczu.

Pierwszym etapem oceny będzie walidacja modeli. W tym celu zastosujemy pięciokrotną walidację krzyżową Monte Carlo za każdym razem dzieląc zbiór na dwie grupy: 70% - ucząca, 30% - testowa. Następnie uśredniamy otrzymane wyniki i dla nich sporządzamy macierz pomyłek oraz krzywą ROC wraz z polem pod krzywą AUC.

5.6.1. Macierz pomyłek i miary klasyfikacji

Podstawą oceny modeli klasyfikacyjnych jest macierz pomyłek, która zestawia rzeczywiste klasy obserwacji z klasami przewidzianymi przez model. Dla problemu binarnej klasyfikacji wyróżnia się cztery podstawowe wartości:

- *TP* (True Positive) – liczba poprawnie przewidzianych przypadków pozytywnej klasy,
- *TN* (True Negative) – liczba poprawnie przewidzianych przypadków negatywnej klasy,
- *FP* (False Positive) – liczba błędnie przewidzianych przypadków pozytywnej klasy,
- *FN* (False Negative) – liczba błędnie przewidzianych przypadków negatywnej klasy.

Na podstawie macierzy pomyłek definiuje się następujące miary jakości klasyfikacji:

Dokładność (Accuracy) określająca odsetek poprawnych klasyfikacji:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (22)$$

Precyzja (Precision) określająca, jak wiele przewidzianych przypadków pozytywnych jest rzeczywiście poprawnych:

$$Precision = \frac{TP}{TP + FP} \quad (23)$$

Czułość (Sensitivity, Recall) określająca zdolność modelu do wykrywania pozytywnych przypadków:

$$Sensitivity = \frac{TP}{TP + FN} \quad (24)$$

Specyficzność (Specificity) określająca zdolność modelu do poprawnego wykrywania przypadków negatywnych:

$$Specificity = \frac{TN}{TN + FP} \quad (25)$$



Negatywna wartość predykcyjna (NPV) określająca poprawność przewidywań klasy negatywnej:

$$NPV = \frac{TN}{TN + FN} \quad (26)$$

Miary te pozwalają na kompleksową ocenę jakości modeli, uwzględniając zarówno poprawne klasyfikacje, jak i błędy różnych typów.

5.6.2. Krzywa ROC oraz pole pod krzywą AUC

Krzywa ROC (Receiver Operating Characteristic) przedstawia zależność pomiędzy czułością modelu a odsetkiem fałszywych alarmów (False Positive Rate):

$$FPR = \frac{FP}{FP + TN} \quad (27)$$

Krzywa ROC jest wykresem funkcji:

$$ROC = (FPR, Sensitivity) \quad (28)$$

Analiza krzywej ROC pozwala ocenić kompromis pomiędzy czułością a specyficznością modelu dla różnych progów decyzyjnych.

Pole pod krzywą ROC (AUC – Area Under Curve) stanowi zagregowaną miarę jakości klasyfikatora:

$$AUC = \int_0^1 TPR(FPR) d(FPR) \quad (29)$$

gdzie TPR oznacza czułość modelu.

Wartość AUC przyjmuje wartości z przedziału $[0, 1]$, gdzie wartość 1 oznacza idealny klasyfikator, natomiast wartość 0,5 odpowiada klasyfikatorowi losowemu. W niniejszej pracy miara AUC została wykorzystana do porównania skuteczności analizowanych modeli klasyfikacyjnych.



6. Rezultaty

6.1. Sieci neuronowe

		Predicted		
		Blue Wins	Red Wins	
Actual	Blue Wins	1056	386	Sensitivity 73.03%
	Red Wins	390	1075	Specificity 73.58%
		Precision 73.23%	Negative Predictive Value 73.38%	Accuracy 73.31%

Fig. 19. Macierz pomyłek dla metody sieci neuronowych.

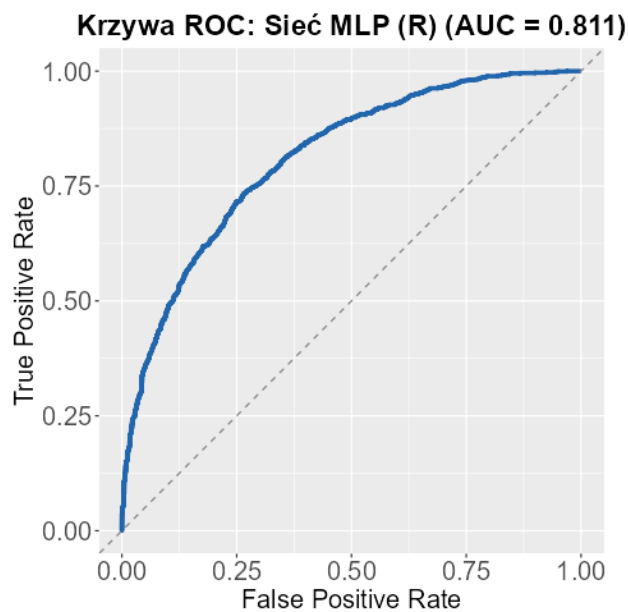


Fig. 20. Wykres krzywej ROC dla metody sieci neuronowych.



6.2. Drzewo klasyfikacyjne

		Predicted		
		Blue Wins	Red Wins	
Actual	Blue Wins	979	467	Sensitivity 74.96%
	Red Wins	327	1134	Specificity 70.83%
		Precision 67.7%	Negative Predictive Value 77.62%	Accuracy 72.69%

Fig. 21. Macierz pomyłek dla metody drzewa klasyfikacyjnego.

Krzywa ROC: Drzewo klasyfikacyjne (AUC = 0.782)

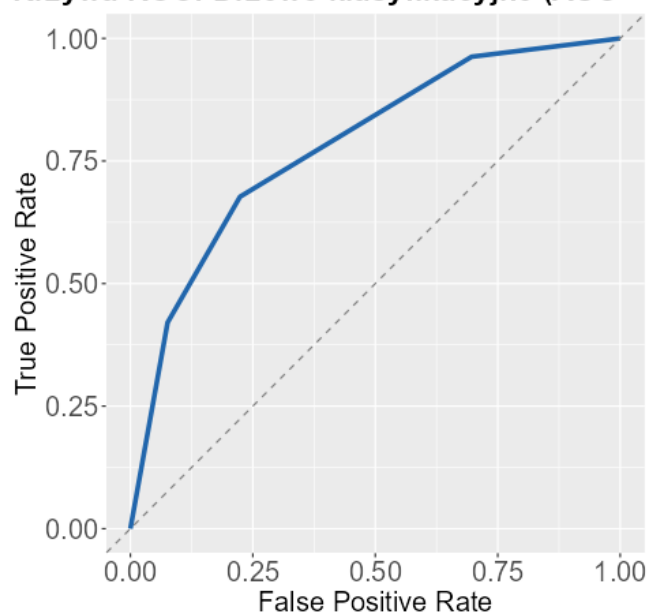


Fig. 22. Wykres krzywej ROC dla metody drzewa klasyfikacyjnego.

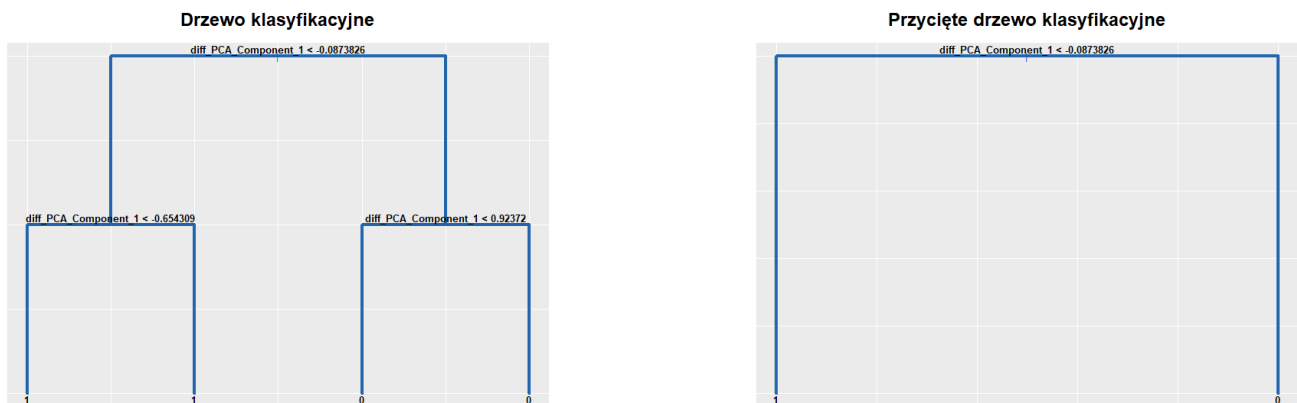


Fig. 23. Wkres normalnego oraz przyciętego drzewa klasyfikacyjnego.

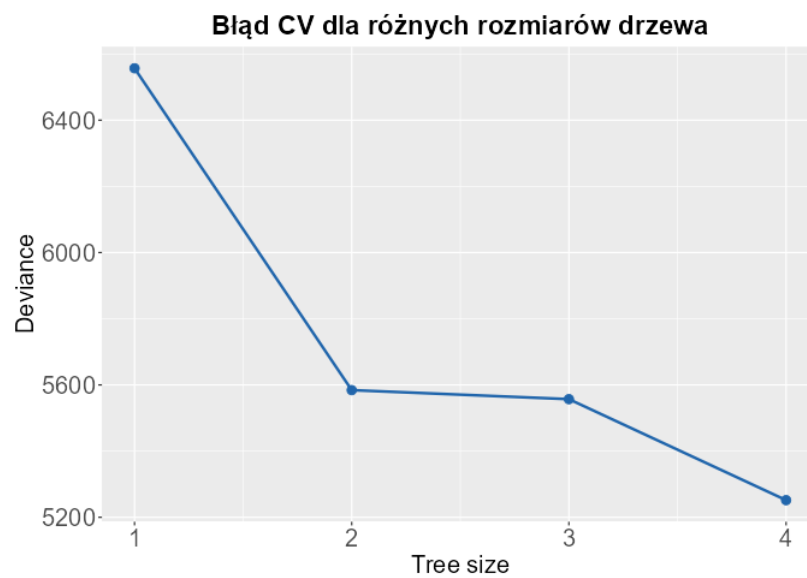


Fig. 24. Wykres błędu walidacji krzyżowej w zależności od rozmiaru drzewa.

6.3. K najbliższych sąsiadów

		Predicted		
		Blue Wins	Red Wins	
Actual	Blue Wins	1007	387	Sensitivity 69.64%
	Red Wins	439	1074	Specificity 73.51%
		PrecisionI 72.24%	Negative Predictive Value 70.98%	Accuracy 71.59%

Fig. 25. Macierz pomyłek dla metody dla metody KNN.

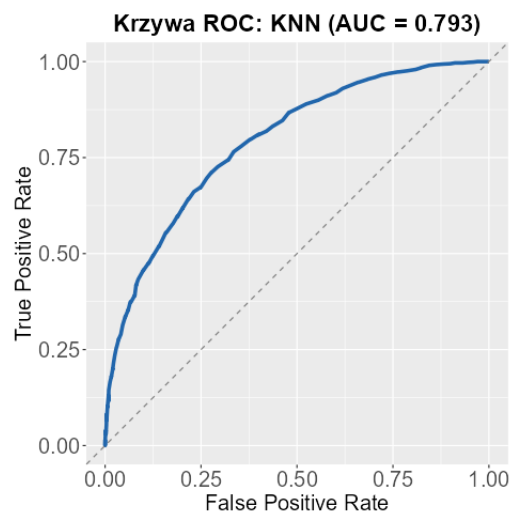
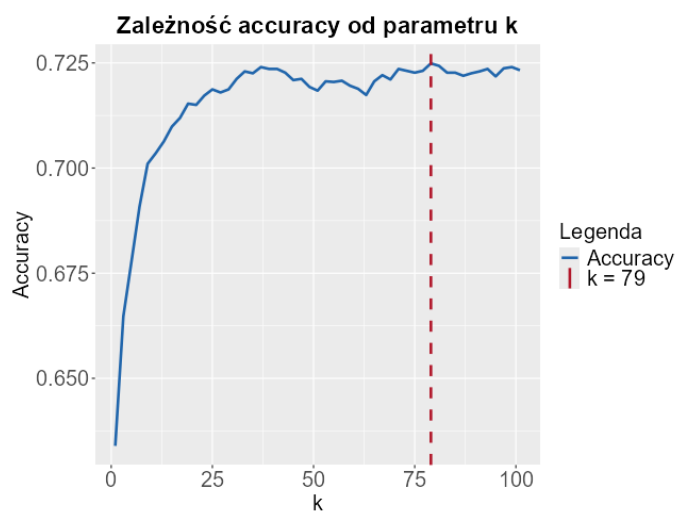


Fig. 26. Wykres accuracy dla różnych wartości k oraz wykres krzywej ROC dla metody KNN.



6.4. Metoda hybrydowa na bazie średniej arytmetycznej

		Predicted		
		Blue Wins	Red Wins	
Actual	Blue Wins	1048	388	Sensitivity 72.48%
	Red Wins	398	1073	Specificity 73.44%
		Precision 72.98%	Negative Predictive Value 72.94%	Accuracy 72.96%

Fig. 27. Macierz pomyłek dla metody mieszanej na bazie średniej arytmetycznej.

Krzywa ROC: Model hybrydowy śr. (R) (AUC = 0.808)

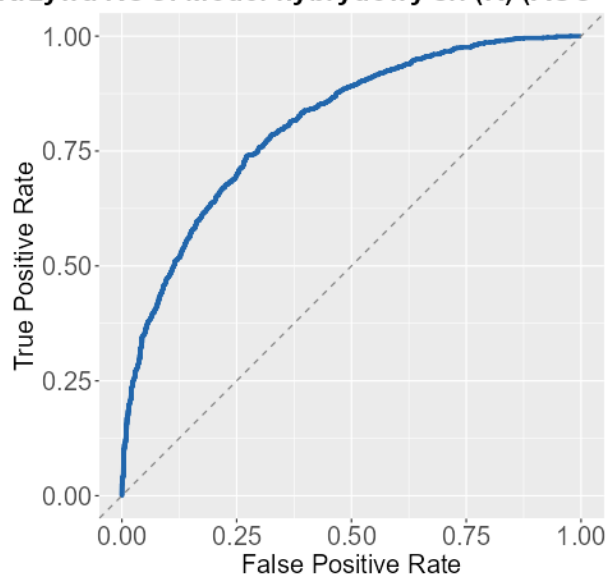


Fig. 28. Wykres krzywej ROC dla metody mieszanej na bazie średniej arytmetycznej.



6.5. Metoda hybrydowa na bazie sieci neuronowej

		Predicted		
		Blue Wins	Red Wins	
Actual	Blue Wins	332	142	Sensitivity 75.97%
	Red Wins	105	293	Specificity 67.36%
		Precision 70.04%	Negative Predictive Value 73.62%	Accuracy 71.67%

Fig. 29. Macierz pomyłek dla metody mieszanej na bazie sieci neuronowej.

Krzywa ROC: Model hybrydowy MLP(R) (AUC = 0.802)

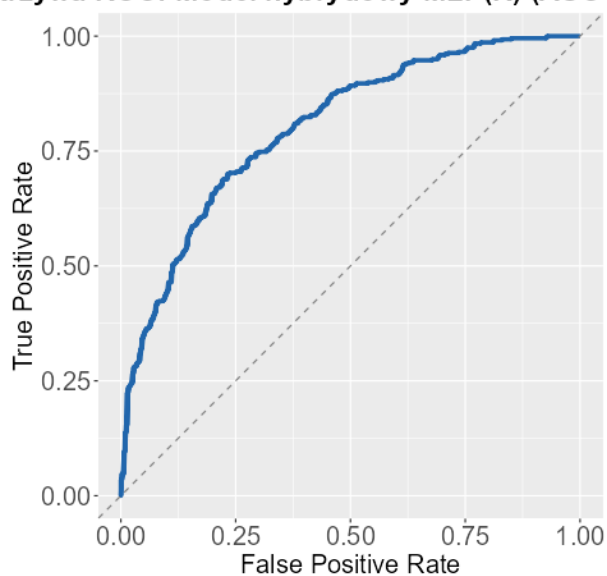


Fig. 30. Wykres krzywej ROC dla metody mieszanej na bazie sieci neuronowej.



6.6. Porównanie skuteczności metod

Model	Accuracy	AUC
Sieci Neuronowe	72,87%	0,806
Drzewo Klasyfikacyjne	72,69%	0,782
KNN	71,54%	0,795
hybr. - Sieć neuronowa	72,48%	72,48
hybr. Średnia	72,65%	0,805

Table 7. Rezultaty modeli na danych testowych.

6.7. Użycie modeli na syntetycznych danych

W celu sprawdzenia działania modeli na nowych obserwacjach wygenerowano syntetyczny zbiór danych z wykorzystaniem pakietu `synthpop` w języku R. Do syntezy zastosowano metodę CART (*Classification and Regression Trees*), która modeluje zależności między zmiennymi za pomocą drzew decyzyjnych i na ich podstawie generuje nowe rekordy. Wygenerowany zbiór zawierał 9689 syntetycznych obserwacji. Parametr `cart.minbucket = 10` ogranicza nadmierne dopasowanie modelu, natomiast `seed = 67` zapewnia powtarzalność wyników. Otrzymany zbiór syntetyczny został wykorzystany jako dane testowe dla opracowanych modeli predykcyjnych.

Model	Accuracy	AUC
Sieci Neuronowe	72,82%	0,804
Drzewo Klasyfikacyjne	71,49%	0,769
KNN	71,47%	0,789
hybr. - Sieć neuronowa	72,48%	0,801
hybr. - Średnia	72,65%	0,800

Table 8. Rezultaty modeli na danych syntetycznych.

7. Podsumowanie

Celem analizy było:

1. zweryfikowanie skuteczności metod klasyfikacji poprzez porównanie predykcji modeli z rzeczywistymi wynikami meczów na bazie danych z pierwszych 10 minut rozgrywki,
2. weryfikacja charakterystyk drużyn wygranych i przegranych oraz określenie, które zmienne mają największy wpływ na końcowy rezultat meczu.

Upřednio przedstawione wyniki skuteczności zastosowanych metod wskazują, że udało nam się osiągnąć dokładność na poziomach odpowiednio: sieć neuronowa MLP = 72,87%, drzewo klasyfikacyjne = 72,69%, KNN = 71,54%, model hybrydowy oparty na średniej arytmetycznej = 72,65% oraz model hybrydowy oparty na sieci neuronowej = 72,48%. Najlepszy wynik uzyskała sieć neuronowa, osiągając dodatkowo wartość AUC równą 0,806. Stanowi to dobry rezultat, potwierdzający, że dane z wczesnej fazy meczu zawierają istotną informację predykcyjną.

Również w odniesieniu do innych prac na tym samym zbiorze danych, jak *LoL: How to Win* [2], osiągnęliśmy porównywalnie dobry wynik, co autorzy korzystający z innych metod klasyfikacji. Dodatkowo nasza najskuteczniejsza metoda przewyższa wyniki uzyskane w analizie skupień na tym samym zbiorze danych, gdzie celność k-means wyniosła 72,46%, a model mieszanin Gaussa estymowany algorytmem EM — 70,88%. Potwierdza to, że zastosowanie dedykowanych klasyfikatorów pozwala na skuteczniejsze przewidywanie wyniku meczu niż metody nienadzorowane.



Wyniki przedstawione w formie tabelarycznej i graficznej, w tym macierz korelacji uwzględniająca zmienną *blueWins*, potwierdzają, że zwycięstwo jest skorelowane znacząco z:

- dużą ilością zabójstw i asyst drużyny niebieskiej, wyrażoną zagregowaną cechą *blueKA*,
- małą ilością zabójstw i asyst drużyny przeciwnej, wyrażoną cechą *redKA*,
- przewagą ekonomiczną i doświadczeniową, odzwierciedloną w zmiennej *diff_PCA_Component_1*,
- kontrolą neutralnych obiektów mapy, takich jak smoki i heroldy,
- wyższą efektywnością farmienia, mierzoną m.in. liczbą zabitych minionów z dżungli oraz wartością *CSPerMin*.

Nasza analiza klasyfikacyjna potwierdza ogólnie przyjęte założenie, że do wygranej potrzeba dużo zabójstw, mało śmierci i dużo asyst, co dodatkowo uzupełnia analizy dokonane w innych publikacjach co do głównych determinant wygranej bądź porażki na podstawie danych z początku rozgrywki [1–3]. Dodatkowo pokazuje, że metody klasyfikacji zapewniają zadowalające odzwierciedlenie faktycznego rezultatu meczu. Stabilność uzyskanych wyników potwierdza również test na danych syntetycznych, gdzie sieć neuronowa osiągnęła dokładność 72,82%, co jest wartością zbliżoną do wyniku uzyskanego na zbiorze testowym.

Załączniki

Poniżej znajduje się lista załączonych plików do projektu wraz z ich opisami:

- **lol.csv** — Badany zbiór danych.
- **ClassificationTree.R** — Implementacja drzewa klasyfikacyjnego wraz z oceną i wizualizacją wyników.
- **CorrelationWisualization.R** — Analiza i wizualizacja korelacji oraz usuwanie silnie skorelowanych zmiennych.
- **endDatasetWisualization.R** — Generowanie wykresów opisujących cechy końcowego zbioru danych.
- **KNN.R** — Implementacja klasyfikatora KNN z doбором parametru *k* i oceną modelu.
- **MixedModel.R** — Implementacja modelu mieszanego na bazie średniej arytmetycznej oraz jego ocena.
- **MixedModelNeural.R** — Implementacja modelu mieszanego na bazie sieci neuronowych oraz jego ocena.
- **PCA.R** — Redukcja wymiarowości danych metodą PCA wraz z wizualizacją wyników.
- **PreliminaryAnalysis.R** — Wstępne przygotowanie i przetwarzanie danych.
- **ROC.R** — Generowanie krzywych ROC i obliczanie wartości AUC.
- **TableWisualization.R** — Wizualizacja macierzy pomyłek i miar jakości klasyfikacji.
- **tensorflow.R** — Implementacja i trenowanie sieci neuronowej z oceną wyników.

Literatura

- [1] Mehdi Gasmi, *League of Legends: what to do in first 10 min*, Kaggle Notebook, 2020. Dostęp: <https://www.kaggle.com/code/servietsky/league-of-legends-what-to-do-in-first-10-min>
- [2] Xiyue Wang, *LoL: How to Win*, Kaggle Notebook, 2020. Dostęp: <https://www.kaggle.com/code/xiyuewang/lol-how-to-win>
- [3] Jeremy Arancio, *Which decisions before 10min lead to win?*, Kaggle Notebook, 2022. Dostęp: <https://www.kaggle.com/code/jeremyarancio/which-decisions-before-10min-lead-to-win>



- [4] Albina Jegorowa, Jarosław Kurek, Michał Kruk, Jarosław Górski, *The Use of Multilayer Perceptron (MLP) to Reduce Delamination during Drilling into Melamine Faced Chipboard*, *Forests*, vol. 13, no. 6, 2022, art. 933. Dostęp: <https://doi.org/10.3390/f13060933>
- [5] Krzysztof Gajowniczek, Marcin Dudziński, *Influence of Explanatory Variable Distributions on the Behavior of the Impurity Measures Used in Classification Tree Learning*, *Entropy*, vol. 26, no. 12, 2024, art. 1020. Dostęp: <https://doi.org/10.3390/e26121020>
- [6] Tomasz Ząbkowski *et al.*, *Changing Electricity Tariff—An Empirical Analysis Based on Commercial Customers' Data from Poland*, *Energies*, vol. 16, no. 19, 2023, art. 6853. Dostęp: <https://doi.org/10.3390/en16196853>
- [7] Yi Lan Ma, *League of Legends Diamond Ranked Games (10 min)*, Kaggle, 2020. Dostęp: <https://www.kaggle.com/datasets/bobbyscience/league-of-legends-diamond-ranked-games-10-min>